Hi-DyPa: Practical Multi-User Watermarking for Detection and Tracing in LLM System

***Abstract*—Large Language Model (LLM) watermarking has emerged as a critical approach for preventing LLM misuses, such as generating illegal content, which requires traceability to individuals, even when the LLM users combine their outputs. Recent works, such as the Multi-user Adaptive watermarking scheme, which has the users' and colluders' traceability and has been proven robust under collusion and text modification attacks. While the state-of-the-art multi-user scheme provides provable security and robustness, its detection rate degrades rapidly when the system population grows. Furthermore, their scheme is vulnerable to the framing attack, where colluders can reconstruct watermarks to accuse innocent users. Their work also lacks an empirical evaluation of the collusion resistance and practical implementation.**

We introduce Hi-DyPa, a Hierarchical Dynamic Parallel watermarking scheme that supports scalable multi-user traceability and addresses existing limitations while preserving all existing watermarking security properties. Our framework introduces a two-level structure with a parallel watermark embedding mechanism and a dynamic codeword generation. Our scheme significantly reduces both time and storage compared to the baseline multi-user scheme and supports large user extensions. The dynamic codeword generation eliminates the framing and colluding attacks with a comprehensive analysis across different collusion scenarios.

We provide the open-source implementation of our Hi-DyPa framework. We evaluate Hi-DyPa on 300 prompts drawn from ELI5, FLAN, and GPT-WritingPrompts using GPT-2 and Deepseek-7B models in a flexible multi-user setting. Experimental results show that Hi-DyPa reduces tracing complexity compared to prior multi-user frameworks and achieves up to approximately 2$\times$ faster tracing for most hierarchical configurations, while maintaining strong attribution performance. In particular, Hi-DyPa attains up to 91.3% full identity accuracy with a false positive rate as low as 3.0% on clean text and reduces the number of wrongly accused users under both two- and three-user collusion.

I. Introduction

The rapid advancement of Large Language Models (LLMs) enables their wide applications across various domains ranging from simple chatbots to advanced content creation tasks [1]. Despite the advancements of the LLMs, the misuse of LLMs can raise challenges in content authentication, intellectual property protection, provenance identification and accountability [2], [3], [4]. LLM watermarking has emerged as a critical approach for addressing these challenges. In particular, a watermarking system embeds watermarks which enable users to verify whether the content, including image, video, or text, was generated by an AI model without requiring access to the original model [5]. It also provides content creators and service providers with proof of ownership through digital signatures, enabling the detection of unauthorized redistribution [6], [4], and supporting the backtrace to identify the specific user who generated the original content for provenance identification and accountability [7], [8].

An effective watermarking system must satisfy all properties: *Robustness* against modifications, *Undetectability* to prevent being legible for adversaries, *Soundness and Completeness* to guarantee recovery, *Traceability* to ensure identification of LLM misuse users and *Efficiency* to support large user satisfaction [8]. However, existing watermarking techniques often fall short of meeting all these requirements simultaneously. For instance, token-bias-based methods [9] and semantic-invariant watermarking [10] demonstrate robustness against multiple attacks but face efficiency challenges. Even industry solutions, like SynthID [5], struggle to be efficient for millions of users while maintaining proper accuracy [11], [12].

Watermarking systems must also withstand adversarial attacks that target these properties. Some attacks, such as text modification attacks [13], emoji insertion evasion [14], and collusion attacks, where multiple users combine watermarked outputs to evade tracing [8], have been considered in prior work. Existing methods also provide partial mitigation, including edit-distance-aware detection [15], semantic-invariant embeddings [10], and frameproof codes [16]. Yet, the framing attack, where adversaries collude to forge another user's watermark to falsely accuse them, remains largely unaddressed and unvalidated in prior schemes.

To fulfil the above requirements on real-world scenarios, a recent work introduced a multi-user watermarking scheme [8] that provides cryptographic proofs of collusion resistance for LLM, establishing the security guarantees through the integration of zero-bit watermarking with collusion-resistant fingerprinting codes. However, this scheme faces the challenges of scalability and attack resistant. In particular, the watermark detection rate degrades when the system population grows. For instance, our experiment shows that the flat multi-user scheme degrades detection accuracy drop from 98.92% to 72.28% when user population grows. Additionally, the detection time, tracing time, and user identification storage are not efficient due to its flat structure in the practical context.

Moreover, in the real world, organizations are rarely "flat", which means treating each person as a system-organized unit. They are hierarchical by design: individuals belong to specific division units. This structure allows the organization to govern identity management, permission granting, or incident response. Therefore, a practical multi-user watermarking scheme should align with the hierarchy of an organization.

Our Contributions: In this study, we introduce Hi-DyPa, a

novel Hierarchical Dynamic Parallel watermarking scheme for multiple users participating in an LLM system. Our proposed two-level watermarking framework for both the embedding and tracing stages, with dynamic codeword generation and parallel embedding, addresses the problem of existing multi-user watermarking approaches. The *hierarchical* structure organizes users into groups, reducing storage complexity and enabling faster tracing for large user deployments. Our scheme is *dynamic* due to the dynamic codeword generation, where master keys are generated for both group and user identifications during the embedding stage and dynamically regenerated for the tracing phase, eliminating the need for a static codeword database. Lastly, *Parallel* embedding with distributed group and user bits randomly throughout the text, ensuring proportional coverage of both group and user layers, for robust tracing. In summary:

1. **Hierarchical Dynamic Parallel Watermarking Structure Design:** We introduce the "Hi-DyPa", a novel Hierarchical Dynamic Parallel watermarking system for multiple users participating in an LLM system that addresses limitations in the existing approaches.
2. **Provable Security Guarantees:** We provide the security analysis and scheme validation on our proposed schemes. Our experiments demonstrate that our scheme preserves all core watermarking security properties from the existing schemes.
3. **Improvements in Storage and Time Efficiency:** We show that compared to the multi-user watermarking schemes, our Hi-DyPa scheme achieves a storage reduction and faster tracing. Theoretically, Hi-DyPa reduces tracing time approximately up to $2\times$ faster and $1600\times$ storage compared to the existing MAU multi-user schemes [8]. Compared to Multi-bit scheme [17], our scheme tracing time is $25\times$ faster.
4. **Enhanced Collusion Resistance and Framing Attacks:** We use a dynamic parallel embedding algorithm to prevent collusion attacks in our Hi-DyPa scheme and generate dynamic codewords on-the-fly to reduce the risk of malicious users colluding to frame innocent users, compared to the existing schemes, while still working effectively under collusion scenarios. This makes the *Robustness* of our scheme stronger than the existing schemes. Previous schemes, such as the Multi-bit scheme [17], are resistant to text modification attacks only.
5. **Empirical Study and Experiments on Collusion:** We provide extensive empirical evidence to highlight the collusion and framing resistance of our scheme in various colluding settings compared to the existing MAU scheme [8].

II. Background and Related Work

Watermarking Operation: A typical watermarking scheme (for language models) consists of four algorithms: *Key Generation, Embedding, Detection, and Tracing* [8], [18]. To additionally support multi-user watermarking, Cohen et al. [8] introduce new key components, including *Secret Key, Tracing Key, User Codewords*, and *PRF seeds*. We further detail the usage of these components in Appendix B.

Watermarking Security Properties: An ideal watermarking scheme would satisfy the evaluation of all watermarking security properties. Christ et al. [18] and Cohen et al. [8] proposed the important watermarking properties:

Soundness: A watermarking scheme is sound if the generated text could not be detected as watermarked [18]. This guarantees that human-written or unwatermarked text is not mistakenly detected as AI-generated [8].

Completeness: A watermarking scheme is complete if the watermarked text is detected with high probability [18], [8].

Undetectability: A watermarking scheme is undetectable if it is infeasible to distinguish between outputs of the watermarked model and the original model, even under adaptive querying [18], [8].

Traceability: A watermarking scheme is traceable or has user attribution if it can identify the specific user(s) who generated watermarked-AI content [8]. This property extends beyond binary detection: a watermarking scheme can only detect whether content is AI-generated or not.

Robustness: A watermarking scheme is robust if the embedded watermarks remain detectable and traceable even after adversarial manipulation [8]. A robust watermarking scheme must be resistant to all the following attacks: (1) text modification attacks such as token deletion [15], [19], paraphrasing [15], [20], [21], synonym substitution [15], [22], [23], [24], [25], and LLM-based rewriting [15], [20], [21]; (2) collusion attacks where multiple users combine their watermarked outputs to evade attribution [8], [26]; and (3) framing attacks where adversaries attempt to falsely implicate innocent users by reconstructing their watermarks.

We define new watermarking properties to evaluate our proposed schemes and previously existing schemes:

Efficiency: Efficiency of a watermarking scheme is tracing time efficiency. We stress the time for watermarking tracing must be efficient within an acceptable time for the specific system's requirements or as fast as possible [11], [27]. More details of Efficiency property in §III

We systematically evaluate our three developed schemes (Hierarchical Static scheme§IV-A, Hierarchical Dynamic Scheme§IV-B, and Hi-DyPa scheme§IV-C) and the existing baseline schemes (Zero-bit scheme [18], L-bit scheme [8], Multiple Adaptive User (MAU) scheme [8] and Multi-bit scheme [17]) against the proposed watermarking properties discussed above (Table I).

Zero-bit Watermarking Scheme: Zero-bit scheme [18] embeds a binary bit watermark message into the LLM-generated text, which provides proof of *Soundness, Undetectability*, and *Completeness* in their work. However, the scheme outputs only watermarked or unwatermarked decisions, without the *Traceability* needed to determine the specific user who generated the text using an LLM. To achieve the *Robustness* and *Efficiency*, a watermarking scheme must achieve the *Traceability*. Since the

TABLE I: Comparison of Baseline Watermarking scheme [18], [8], [17] with our Proposed Schemes§IV-A, §IV-B, §IV-C

	Zero-bit [18]	**L-bit** [8]	**MAU** [8]	**Multi-bit** [17]	**Hierarchical Static[§IV-A]**	**Hierarchical Dynamic [§IV-B]**	**Hi-DyPa [§IV-C]**
Soundness	✓	✓	✓	✓	✓	✓	✓
Undetectability	✓	✓	✓	✓	✓	✓	✓
Completeness	✓	✓	✓	✓	✓	✓	✓
Traceability	✗	✓	✓	✗	✓	✓	✓
Efficiency [Tracing Time]	✗	✗	✗	✗	✓	✓	✓
Robustness [Text Modifications & Collusion & Framing Attacks]	✗	✗	✗	✗	✗	✗	✓

Zero-bit scheme is not traceable, it cannot achieve *Robustness* and *Efficiency* (Table I).

L-bit Watermarking Scheme: L-bit scheme [8] embeds an L-bit message by applying the underlying Zero-bit scheme, inheriting *Soundness, Undetectability*, and *Completeness*. This scheme achieves the *Traceability* by enabling a single user identifier to be recovered from the watermarked text. However, this scheme has no *Robustness* against collusion and framing attacks. The scheme does not guarantee *Efficiency* due to the slow tracing of the flat design (Table I).

Multiple Adaptive User (MAU) Watermarking Scheme: MAU [8] composes the L-bit scheme with collusion-resistant fingerprinting codes [28], [29], [30], thereby preserving *Soundness, Undetectability, Completeness, Traceability* and being resistant to collusion attack. Similar to L-bit scheme, MAU scheme does not ensure *Efficiency* and *Robustness* because of not support being resistant to framing attack (Table I).

Multi-bit Watermarking Scheme: Multi-bit Watermarking Scheme [17] embed longer single-user messages, achieving *Soundness, Undetectability* and *Completeness*. However, the scheme is not designed for multi-user attribution and provides no mechanism against collusion or framing; thereby, it satisfies neither *Efficiency* nor *Robustness* (Table I).

More details of the discussed existing watermarking schemes above in Appendix A.

Existing Watermarking Notations: We denote the security parameter as λ, and for n users in the system with $n \in \mathbb{N}$. String concatenation is denoted by $\|$. We use $\perp$ to denote the failure of an operation, such as the failure of watermarking message extraction. A function $\mathrm{negl}(\lambda)$ is *negligible* if it goes to 0 faster than any inverse polynomial in λ. We use $\mathrm{poly}(\cdot)$ for an unspecified polynomial.

Let $\mathcal{T}$ be the token alphabet or vocabulary of the language model, and let $\mathcal{T}^*$ denote finite token sequences or strings. A generated text is a string $T \in \mathcal{T}^*$ with length $|T|$. We write $T[i]$ for the i-th token, and use standard substring notation $T_{i:j}$ for tokens i through j. We use ϵ for the empty string. For $L \in \mathbb{N}$, $\{0,1\}^L$ denotes length-L bitstrings. When comparing bitstrings of equal length, we use a similarity score based on Hamming agreement distance (see Appendix C).

A fingerprinting code is a pair $\mathrm{FP} = (\mathrm{FP.Gen}, \mathrm{FP.Trace})$. $\mathrm{FP.Gen}(1^\lambda, n)$ outputs a code matrix $X \in \{0,1\}^{n \times L}$ and a tracing key tk. The n-th user is assigned codeword $X[n] \in \{0,1\}^L$. Given an extracted watermark message $\widehat{m}$ and tk, the tracing algorithm outputs a set of accused users $\mathrm{FP.Trace}(\widehat{m}, \mathsf{tk}) \subseteq [n]$.

Hierarchical Watermarking Notations: The system organizes $n = G \times s$ users into G groups of s users. Codeword length $L = L_g + L_u$ splits between group and local layers. The split ratio $r \in (0,1)$ controls block allocation in parallel embedding. Group codewords $X_g[g] \in \{0,1\}^{L_g}$, with group seeds $K_g[g] \in \{0,1\}^\lambda$, and position templates $T_u[u] \in \{0,1\}^\lambda$. User (g,u) has a composite codeword with the combination of the group codeword C_g and user codeword C_u. Watermark message extraction from suspect text $\hat{T}$ yields $\hat{m}$, split into $\hat{m}_g = \hat{m}[1 : L_g]$ and $\hat{m}_u = \hat{m}[L_g + 1 : L]$, compared against thresholds τ_g (for group) and τ_u (for user within group). The final accused group set and the accused user set will be denoted as g^* and u^*, respectively.

Limitations: At a high-level, our proposed scheme addresses the following efficiency and robustness limitations of the existing watermarking schemes (see Table I):

Framing Vulnerability: Existing watermarking schemes that support multiple users [8] are vulnerable to the framing attack, where adversaries can collude to frame innocent users. When users collude and have access to their individual codewords (see Appendix D2), the feasible colluding codeword set they produce may contain codewords that are closer to an innocent user's codeword than to any colluder's codeword (see Appendix D1). This vulnerability occurs in static codeword schemes because the codewords are statically stored in the system database instead of being generated for the specific watermarking operations.

Large Codeword Storage: The multi-user MAU scheme requires storing an individual fingerprinting codeword per user, causing storage complexity to grow linearly with the number of users. It would be beneficial for computational savings if we could improve this feature.

Slow Embedding, Detecting, and Tracing: The tracing algorithm in MAU scheme requires comparing the extracted message against all registered user codewords. This results in linear tracing time, which becomes computationally expensive for large-scale deployments.

No empirical experiments on the collusion and framing attacks: Cohen et al.'s [8] provides strong theoretical foundations with formal proofs of collusion resistance based on fingerprinting code properties. However, the study lacks empirical experiments on collusion attacks.

A. System Architecture

The design idea of the Hi-DyPa watermarking scheme originated from the divide-and-conquer principle in large, real-

world organisations, analogous to a hierarchical B-tree structure [31]. Our system design can be applied to a corporation that organizes employees into multiple departments or groups with unique identification for later tracing. Instead of searching through all system users sequentially, the framework first identifies the group that contains the users who misuse the LLM, then searches only within that group to find the accused users.

The system operation involves key participants within a hierarchical structure: The LLM users, the system administrator and the authorized detectors who are trusted parties. Users are organized into groups depending on the organization's purposes. When users interact with the system by submitting queries to the LLM, each is uniquely identified by a composite codeword, which includes both the group code and the local user code within each group (see Figure 1). The LLM receives user queries and generates candidate text responses. The system administrator holds the secret key, which will be used for watermark embedding and detection. This key will enable the pseudorandom function that determines token biasing during generation.

Embedding stage (Figure 1): The algorithm randomly embeds the composite codeword watermark, consisting of both group bits and user bits, into high-entropy blocks of generated text. The group codeword bits identify the group identity, and the local codeword bits identify the user identity within the group. The secrecy of the key (master secret key, see Appendix B) ensures that the embedded watermarks are undetectable to unauthorized parties while remaining extractable by authorized detectors. The scheme embedding algorithm receives three inputs (LLM-generated text, user's composite codeword, and the secret key) and outputs the watermark text. Parallel embedding distributes group and user watermark bits randomly throughout the generated text, ensuring proportional coverage of both hierarchical layers.

Tracing stage (Figure 2): Given the inputs (watermarked text, secret key and tracing key) (see Appendix B), the detector extracts the embedded message. It evaluates whether enough watermark bits are present in both the group portion and the user portion of the recovered codewords for group tracing and user tracing. For static hierarchical scheme, the algorithm first identifies the group by comparing extracted group-layer bits against stored group codewords in the system database. Then identifies the accused user within that group by comparing the extracted user codeword portion to the user codeword database. For dynamic hierarchical scheme, the tracing algorithm is operated sequentially for group and user, similar to the static hierarchical scheme; however, the codeword will be regenerated dynamically for group comparison, then user comparison. The trusted parties must be authorized to conduct the tracing investigation. These are those who require the identification of the misused LLM-generated content. The trusted parties must be authorized to access the master keys to prevent unauthorized surveillance.

B. Threat Model

The detectors have access to the system master keys for conducting watermark detection and tracing. The adversaries are the system users who may attempt to: (1) avoid detection by corrupting or removing the embedded watermarks, (2) avoid tracing by being detected as watermarked while preventing accurate user identification, and (3) framing innocent users by colluding to manipulate watermarked outputs that falsely include the innocent's codewords. Specifically, our scheme aims to address the following black-box attacks:

1) Text Modification Attacks.: These attacks attempt to remove or degrade embedded watermarks while not impacting the semantic content of the generated text [13]. Specifically, we consider the attackers can launch the following attacks:

Deletion Attacks: The adversary partially removes tokens from watermarked text, reducing codeword watermark-carrying positions for detection [19], [15].

Paraphrasing Attacks: The adversary uses model-based paraphrasers to rephrase watermarked text while preserving semantic meaning [20], [15], [21]. Paraphrasing modifies the token distribution, disrupting embedded watermark codewords.

Synonym Substitution: The adversary performs lexical replacements while preserving sentence structure [22], [15], [23], [24], [25]. This attack targets individual high-entropy positions where watermark bits are embedded. Most codeword bits embedded into the high-entropy text block will cause the watermark to be destroyed if the text blocks are lexically substituted.

LLM-based Rewriting: The adversary uses a language model to regenerate text while preserving original meaning [20], [15]. This is the strongest semantic-preserving transformation, as regenerated text may share minimal token overlap with the original.

2) Collusion Attacks: We consider following scenarios:

Same-group Collusion: This scenario is analogous to the flat multi-user colluding scenario [8]. Users in the same group share the identical group codeword portion but have different local codewords. When users from the same group collude, the group-level watermark remains the same, but the user-level watermark is diluted via colluding depending on the individuals' codewords and the number of colluders [30].

Cross-group Collusion: Users from different groups collude. In this scenario, both group-level and user-level watermarks can be mixed. This allows the malicious colluders to conduct the framing attacks (see §II-B3) if they have the knowledge of other users' codewords or study the codeword pattern over time.

3) Framing Attacks: The framing attack occurs when the adversaries attempt to construct the codewords via collusion, which makes the tracing algorithm fail [32]. This results in the innocent users being accused, especially when the adversaries have the knowledge of other users' codewords by accessing the static codeword database or guessing others' codeword patterns.

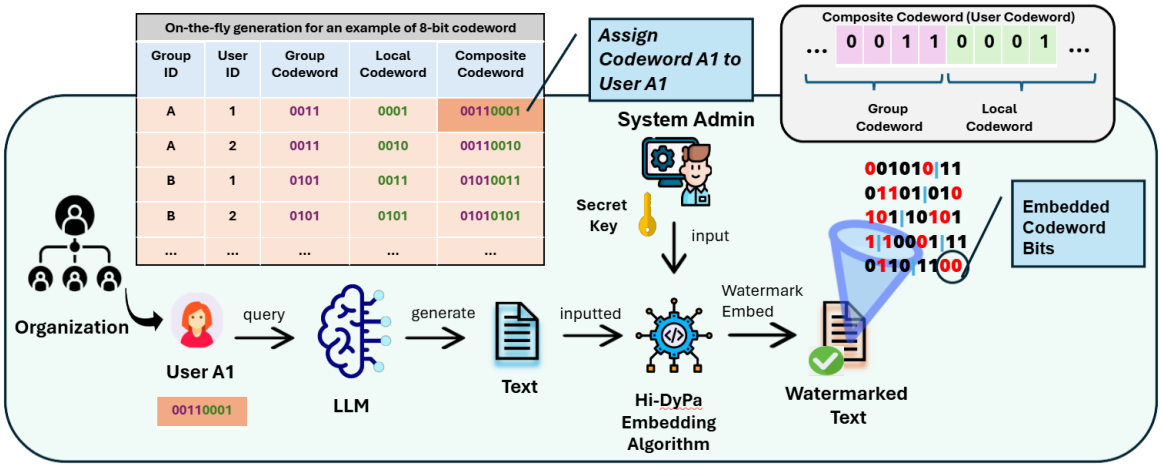

Fig. 1: Hi-DyPa scheme: Watermark Generation, Embedding Stage and Key Management.

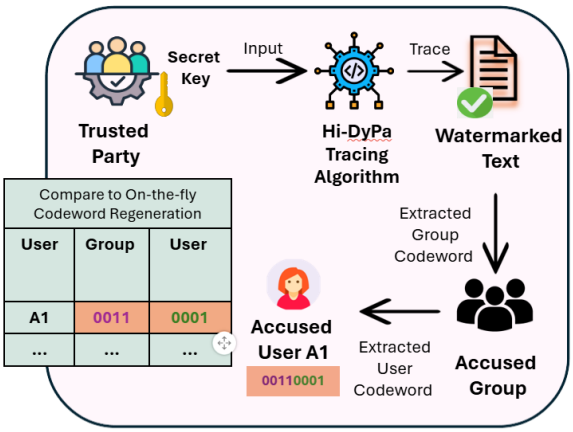

Fig. 2: Hi-DyPa scheme: Watermark detecting and tracing. All the extracted group or user codewords are compared to the on-the-fly codeword generation. The trusted parties can access the watermarking secret key from the system admin if they are authorized to trace the LLM-misuse users.

We present the collusion attack and framing attack strategies in more detail in Appendix D1 and D2.

Out-of-scope attacks. We exclude white-box attacks that allow the adversaries to have full access to system internals, such as watermarking architecture, algorithm implementation, or the secret key [33], [34]. With the internal watermarking system knowledge, the adversary can identify and remove watermark-carrying tokens without degrading text quality [9], [13], [14], [12]. The admin-as-adversaries scenario is considered a white-box attack where the administrator leaks the keys and codeword database to unauthorized parties [35] or conducts frame attacks with the knowledge of user codeword storage. However, these internal attacks can be mitigated by insider threat controls, such as key management distribution. Critical components, such as keys, and codeword database, can be distributed or rotated across multiple personnel [36].

III. Efficiency in Hierarchical Watermarking System

We define a watermarking scheme as time-efficient if and only if the tracing time does not exceed the latency determined by the system's deployment requirements. Let Ψ denote the acceptable tracing latency, set by operational policy.

Definition III.1 (Tracing Time Efficiency)**.** A traceable watermarking scheme $\mathcal{W}$ is *tracing-time-efficient* with respect to Ψ if and only if

$$T_{\text{trace}}(n, L) \leq \Psi. \tag{1}$$

Specifically, *Efficiency* in a hierarchical watermarking scheme can be determined by the layer-wise choice, which is the (group, user) set choice to enhance the tracing time of the system (more details see VII).

Additionally, *Efficiency* in a hierarchical watermarking scheme can be considered efficiency in other watermarking operations such as key generation, embedding and detection and efficiency in system storage. We include these efficiency evaluations in our extended experimental results in Appendix I1.

IV. Proposed Schemes

A. Hierarchical Static scheme

We first describe a *baseline hierarchical static* scheme, which serves as a conceptual starting point and motivates the need for dynamic, group-specific user codewords. We consider Cohen et al.'s multi-user scheme and this basic hierarchical construction as *static* schemes, since all group and user codewords are pre-generated and stored in a system database prior to watermark embedding. The system assigns n users into G groups of s users each. The total codeword length is L, which is split between the group layer (L_g) and the local layer (L_u). In our default configuration, $L_g = L_u = L/2$. Other allocations for the ratio between group-level and user-level codewords' length, depending on specific system purposes,

will be discussed in Appendix H. For a specific user (g, u), the composite codeword is formed as $C = C_g \| C_u$, where $C_g = X_g[g]$ is the group codeword and $C_u = X_u[u]$ is the local codeword. Each layer operates as an independent watermarking scheme with its own codeword and decoding procedure. The hierarchical static scheme $\mathcal{W}_{\text{hier-sta}}$ and each layer of its construction are in Appendix E1.

Key Generation (see algorithm 1 in Appendix E2): The algorithm invokes the underlying multi-user watermarking scheme's key generation to produce the secret key sk. This key enables the pseudorandom function that determines token biasing during text generation. At the group layer, the system instantiates a codeword X_g for group identifiers together with an associated tracing key tk. However, differences in group-level codewords with shared local codewords do not yield attribution guarantees. For each user position $u \in [s]$ within a group, the algorithm generates a random L_u-bit string local codeword. These local codewords are shared across all groups—meaning user position 1 in Group A has the same local codeword as user position 1 in Group B (see Appendix D2).

Embedding (see algorithm 4 in Appendix E5:) Given the user's identity (g, u), the algorithm retrieves the group codeword $X_g[g]$ and local codeword $X_u[u]$. The group and local codewords are concatenated to form the complete composite codeword. The composite codeword is embedded into the generated text using the underlying multi-user watermarking primitive. At each high-entropy position, the PRF parameterized by sk determines a preferred token. The watermarking algorithm biases token selection to embed each bit of C. Only positions with sufficient entropy are used for embedding.

Tracing (see algorithm 7 in Appendix E8): Using the underlying multi-user extraction algorithm, the detector recovers the embedded message $\hat{m}$ from the suspect text. The determination of the accused users will be the result of the Hamming distance comparison function (discuss in Appendix C). If extraction fails ($\hat{m} = \perp$), indicating no detectable watermark, it returns empty. The extracted L-bit message is partitioned into two components: (1) $\hat{m}_g$: The first L_g bits representing the expected group watermark; and (2) $\hat{m}_u$: The remaining L_u bits representing the expecting local user watermark.

Phase 1 – Group-Level Tracing: The algorithm applies the group-layer tracing procedure to the extracted group portion $\hat{m}_g$ using the group tracing key $\mathsf{tk_g}$. The tracing procedure returns a set of accused groups consistent with the group codeword X_g, providing collusion resistance guarantees [37].

Phase 2 – User-Level Tracing: Once a group is identified, the algorithm searches only within that group's s users, using user tracing key $\mathsf{tk_u}$. For each user position u, it computes a similarity score between the extracted local portion $\hat{m}_u$ and the stored local codeword. Our scheme supports partial identification—if group tracing succeeds but user-level tracing fails, the algorithm returns only the group identity.

Static Database Constraints: Because group and user codewords are stored persistently, the static baseline requires protecting the key and codeword storage as part of system operational security (§II-B). Our next scheme reduces this exposure by generating local codewords dynamically rather than storing them permanently (see §IV-B).

Framing Vulnerability: There is a critical vulnerability to cross-group collusion that arises from the shared local codewords across groups. Consider users (g_1, u) from Group g_1 and (g_2, u) from Group g_2 who share the same local position u. The local-level watermarks are identical, providing no distinguishing information. If the tracing algorithm correctly identifies one group (say g_1), it will correctly identify user position u within that group. However, the actual colluder might be user (g_2, u), not (g_1, u). This can accuse the wrong user who shares the same local position. Adversaries can exploit this to frame innocent users with the same position in other groups. If colluders from different groups share the same local position, they can collude to ensure the extracted local codeword perfectly matches an innocent user in a different group, enabling a targeted framing attack. This vulnerability shows that group-level separation alone is insufficient and motivates our subsequent improvement: the Hierarchical Dynamic scheme (see §IV-B).

B. Hierarchical Dynamic scheme

We introduce the Hierarchical Dynamic scheme, which preserves the hierarchical structure of the static baseline while replacing stored codewords with on-the-fly generation using pseudorandom functions. Unlike the static baseline, this scheme is dynamic: codewords are not stored in a system database but are deterministically regenerated during embedding and tracing. Therefore, it eliminates the risk of codeword database leakage. The Hierarchical Dynamic watermarking scheme $\mathcal{W}_{\text{hier-dyn}}$ is derived from the basic hierarchical construction $\mathcal{W}_{\text{hier-sta}}$. The Hierarchical Dynamic scheme assigns each user a group-specific local codeword, eliminating the shared-codeword vulnerability identified in the static baseline. The key insight of this scheme is that we can achieve unique local codewords without storing them explicitly. We leverage pseudorandom function: $C_u = \text{PRF}(K_g[g] \| T_u[u])$ where $K_g[g]$ is a secret seed specific to group g, and $T_u[u]$ is a public template for position u. Different groups have distinct seeds, resulting in different PRF outputs. Without knowing $K_g[g]$, adversaries cannot predict codewords. This scheme still preserves the *Efficiency* since only seeds and templates are stored, not actual codewords. Local and group codewords are instantiated only transiently for embedding or tracing and are not persistently stored. Then, in the tracing process, they will be recreated for comparison with the extracted codewords.

Key Generation (see algorithm 2 in Appendix E3): The algorithm generates a secret seed $K_g[g]$ for each group. For each group $g \in [G]$, the algorithm generates a random λ-bit secret seed $K_g[g]$ which will be used for local codeword generation. For each user position $u \in [s]$, the algorithm generates a random λ-bit template $T_u[u]$. These templates, combined with the secret group seed, produce unique local codewords.

Embedding (see algorithm 5 in Appendix E6): The embedding algorithm of this scheme is similar to the basic Hierarchical scheme; however, instead of retrieving a stored codeword from the database, the algorithm computes the local codeword generation PRF. For users (g_1, u) and (g_2, u) with $g_1 \neq g_2$, since $K_g[g_1] \neq K_g[g_2]$, PRF security implies that users' local codewords in the same local position are computationally independent. For users (g, u_1) and (g, u_2) with $u_1 \neq u_2$, since $T_u[u_1] \neq T_u[u_2]$, the PRF inputs differ, producing distinct outputs. The group and local codewords are concatenated and then embedded into generated text using the underlying L-bit watermarking primitive, similar to the hierarchical static scheme.

Tracing (see algorithm 8 in Appendix E9): The tracing algorithm operates in the same way as the Hierarchical scheme. Once a group is identified, the algorithm searches within that group by regenerating local codewords on-the-fly: $C_u = \text{PRF}(K_g[g^*], \|T_u[u])$. The tracing algorithm regenerates codewords using the accused group's seed $K_g[g^*]$. For cross-group collusion between (g_1, u) and (g_2, u): if accused group $g^* = g_1$, the regenerated codeword PRF does not match the codeword from user (g_2, u). This prevents the cross-group framing attack described in the static baseline.

Sequential codeword vulnerability: While the Hierarchical Dynamic scheme addresses the cross-group collusion vulnerability through dynamic codeword generation, it has the sequential codeword embedding limitation. The composite codeword is embedded by placing all group bits first, followed by all user bits. When the generated text has limited high-entropy blocks, such as short responses or low-entropy content, the embedding may exhaust available blocks before reaching the user layer, resulting in complete group coverage but zero user-bit coverage, making user identification impossible. Besides, text modifications can affect different layers: deleting the beginning of text destroys group bits while preserving user bits, whereas deleting the end destroys user bits while preserving group bits. Since hierarchical tracing requires successful group identification before proceeding to user-level tracing, the loss of group bits causes the entire tracing process to fail even when user bits remain intact. These limitations motivate the Hierarchical Dynamic Parallel (Hi-DyPa) scheme (§IV-C).

C. Hierarchical Dynamic Parallel scheme

We introduce the *Hierarchical Dynamic Parallel* (Hi-DyPa) scheme, which removes the positional fragility caused by sequential embedding in the Hierarchical Dynamic scheme. In Scheme IV-B, group bits are embedded before user bits, creating an ordering dependency that makes attribution sensitive to localized text modifications. Because hierarchical tracing requires successful group identification before user-level attribution, loss of group-level evidence can cause attribution to fail even when user-level signal remains intact. Hi-DyPa eliminates this dependency by embedding group and user bits in parallel across the text. When combined with a minimum-distance constraint on group identifiers (see H3), parallel embedding ensures that group- and user-level evidence degrades proportionally rather than causing abrupt attribution failure.

Key Generation (see algorithm 3 in Appendix E4): Key generation is identical to the Hierarchical Dynamic scheme (Algorithm 2), with one additional parameter: the split ratio r. The split ratio controls the expected proportion of embedding positions allocated to the group and user layers.

Embedding (see algorithm 6 in Appendix E7): During embedding, each high-entropy position is independently assigned to the group or user layer based on the split ratio $r \in (0, 1)$. Values $r > 0.5$ bias embedding toward group bits, while $r < 0.5$ prioritizes user bits. For m high-entropy positions, the expected number of group-bit embeddings is $r \cdot m$, with the remaining $(1 - r) \cdot m$ positions allocated to user bits. As a result, group and user bits are distributed throughout the generation, ensuring that evidence from both layers degrades proportionally under truncation or rewriting.

Tracing (see algorithm 9 in Appendix E10): Tracing follows the Hierarchical Dynamic scheme (Algorithm 8) but requires an additional reconstruction step. Because group and user bits are interleaved, the tracer must first reconstruct which extracted bits correspond to each layer using the candidate group seed $K_g[g]$. The algorithm therefore iterates over candidate groups during extraction, reconstructs the group and user portions of the watermark in parallel, and then applies hierarchical tracing.

V. Theoretical analysis

A. Security analysis

1) Inherited Watermarking Guarantees.: Hi-DyPa inherits the core security properties of the underlying AEB-style watermarking framework [8], as it does not modify the embedding or detection primitives. In particular, watermark embedding continues to rely on PRF-derived token biasing with entropy-aware gating, and detection is performed using the same statistical tests as in [8].

Undetectability. Since Hi-DyPa preserves the embedding and detection mechanisms of the base scheme, any distinguisher against Hi-DyPa directly implies a distinguisher against the underlying watermark. Thus, the undetectability of Hi-DyPa reduces to that of the base AEB-style construction.

Theorem 1 (Undetectability)**.** *Let $\mathcal{W}'$ be an undetectable multi-user watermarking scheme constructed from an undetectable multi-user scheme [8]. Then the Hi-DyPa scheme $\mathcal{W}_{\text{Hi-DyPa}}$ is undetectable.*

Full proof in Appendix F1.

Soundness. Soundness guarantees are likewise preserved. Hierarchical decoding is applied only after recovery of the L-bit watermark message, and user-level attribution is conditioned on correct group identification. As a result, the decoder never forces a specific user attribution when the recovered signal is unreliable, ensuring that false positive rates cannot increase relative to the base detector.

Theorem 2 (Soundness)**.** *Let $\mathcal{W}'$ be a sound multi-user watermarking scheme. If $\mathcal{W}'$ is sound, then the Hi-DyPa scheme $\mathcal{W}_{\text{Hi-DyPa}}$ constructed from $\mathcal{W}'$ is also sound.*

Full proof in Appendix F2.

Completeness. Completeness is maintained whenever a sufficient fraction of watermark-bearing tokens survives post-generation transformations. In such cases, the recovered L-bit message enables correct group identification and, when possible, user identification within the group.

Theorem 3 (Completeness)**.** *Let $\mathcal{W}'$ be multi-user watermarking scheme. Let FP_g be a fingerprinting code for G groups, and let the user-layer use PRF-based codeword generation for s users per group. The Hi-DyPa scheme $\mathcal{W}_{\text{Hi-DyPa}}$ is complete: for any user $(g, u) \in [G]\times[s]$ and any prompt Q with sufficient empirical entropy.*

Full proof in Appendix F3.

Adaptive Robustness. Hi-DyPa inherits the AEB-style robustness guarantees from the underlying scheme, and the parallel embedding mechanism provides additional proof.

Theorem 4 (Adaptive Robustness)**.** *Let $\mathcal{W}'$ be an adaptively R_k-robust multi-user watermarking scheme for some robustness condition R_k as defined in [8]. Then Hi-DyPa is adaptively robust: for any efficient adversary $\mathcal{A}$ that queries $\mathsf{Wat}_{\mathsf{sk}}((g_i, u_i), Q_i)$ obtaining responses T_i, and outputs modified text $\hat{T}$.*

Full proof in Appendix F4.

Collusion Resistance. Hi-DyPa achieves collusion resistance through two mechanisms: (1) the group-level fingerprinting codes [37] provide mathematical guarantees that at least one guilty group is identified from the feasible set of colluded codewords, and (2) the hierarchical structure and dynamic codeword generation reduce the collusion problem as discussed in IV.

Theorem 5 (Collusion Resistance)**.** *Let FP be a robust fingerprinting code secure against c colluders. Let $\mathcal{W}'$ be a robust multi-user watermarking scheme. Then Hi-DyPa is collusion-resistant: for any coalition $\mathcal{T} = \{(g_1, u_1), \ldots, (g_c, u_c)\}$ of at most c users, and any efficient adversary $\mathcal{A}$ that combines watermarked outputs from users in $\mathcal{T}$ to produce $\hat{T}$. That is, at least one colluder is correctly identified with overwhelming probability.*

Full proof in Appendix F5.

Framing Attack Resistance Hi-DyPa addresses this vulnerability through dynamic PRF-based codeword generation, making it computationally infeasible for colluders to predict or reconstruct an innocent user's codeword without knowledge of the victim's group seed.

Theorem 6 (Framing Attack Resistance)**.** *Let $PRF : \{0,1\}^\lambda \times \{0,1\}^* \to \{0,1\}^{L_u}$ be a secure pseudorandom function. In the Hi-DyPa scheme with dynamic codeword generation, for any coalition $\mathcal{T}$ of at most c colluders and any innocent user $(g^*, u^*) \notin \mathcal{T}$, the scheme is framing attack resistant for any adversarial output $\hat{T}$ in the feasible set of $\mathcal{T}$.*

Full proof in Appendix F6.

2) Hierarchical Decoding and Attribution Safety: Hi-DyPa performs attribution using a two-stage hierarchical decoding procedure that prioritizes conservative decisions under noise. After recovering the L-bit watermark message, the decoder first identifies the originating group using the group-level codeword, and performs user-level decoding only if the group is confidently identified; otherwise, it returns a group-only or null attribution. This design avoids forced user attribution when the recovered signal is unreliable. Unlike flat multi-user schemes that must select a single user from the entire population, hierarchical decoding restricts user identification to a smaller candidate set and allows attribution to degrade gracefully as noise increases. As a result, Hi-DyPa reduces misattribution risk and favors safety-oriented outcomes without modifying the underlying watermark embedding or detection mechanisms.

B. Tracing Time Complexity analysis

The *baseline Multi-user scheme* tracing algorithm compares the extracted L-bit message against all n user codewords. Therefore, the total tracing time complexity of the baseline multi-user scheme is: $O(n \cdot L)$. Meanwhile, the *Hierarchical Static Scheme*'s two-phase tracing significantly reduces tracing time complexity. In group-level tracing, the algorithm extracts L-bit message, and then it compares the group portion against G group codewords: $O(G \cdot L_g)$. In user-level tracing, it compares the local portion against s local codewords within the identified group: $O(s \cdot L_u)$. Therefore, the total tracing time of the *Hierarchical Static Scheme* is $O(G \cdot L_g + s \cdot L_u)$, which depends on the chosen partition (G, s), not on n alone as the baseline flat scheme. To show that the *Hierarchical Static scheme* is faster than the *baseline multi-user scheme*, we compare their tracing time complexities by computing the difference between the two and proving that it is always non-negative. The two schemes only have the same tracing time in the worst case of a single group setting. Details of the proof that this difference is always a non-negative number in Appendix G. Both *Hierarchical Dynamic Scheme* and *Hi-DyPa Scheme* require additional PRF calls per user position to regenerate the codewords on-the-fly. Additionally, the *Hi-DyPa Scheme* calls the PRF per high-entropy block to reconstruct the parallel layer assignment. However, these additional PRF calls are negligible because each call is a single HMAC-SHA256, which costs insignificantly compared to bit-comparison and message extraction work that already dominates tracing. Therefore, the tracing time of *Hierarchical Dynamic Scheme* and *Hi-DyPa Scheme* are roughly equal to the *Hierarchical Static Scheme*, and faster than the baseline flat MAU scheme.

The key generation is a one-time system setup cost, and the embedding time is the same across all the discussed schemes due to the underlying L-bit message embedding mechanism.

VI. Experiment and Evaluation

A. Experimental Setup

We empirically evaluate the efficiency, and robustness of Hi-DyPa, as summarized in Table I, using GPT-2 and Deepseek-7B with its standard tokenizer, following common practice in recent watermarking evaluations [38], [39], [17], [13], [40]. Text is generated using stochastic decoding, with generations capped at 512 tokens unless otherwise stated (400 for collusion experiments and 256 for rewrite attacks).

We evaluate on a fixed set of 300 prompts drawn from three domains: 100 explanatory prompts from ELI5 [41], 100 instruction-following prompts from FLAN [42], and 100 open-ended creative prompts from GPT-WritingPrompts [43]. This prompt set captures diverse generation styles and aligns with prior multi-prompt evaluation protocols [44], [11].

Watermarking follows an AEB-style framework with entropy-gated embedding and PRF-based token scoring [8]. We evaluate two attribution schemes: *Hi-DyPa*, the final hierarchical scheme proposed in this work, and a flat multi-user MAU baseline [8] on their robustness: evaluated on text modifications, collusion and framing attacks. For experiments on tracing time efficiency only, we evaluated the tracing time of our proposed scheme compared to the existing works, including: [45], [40], [46], [17] and [8]. All experiments fix the watermark length to $L = 8$ and use shared parameters ($\delta = 3.5$, $\tau = 2.0$), and evaluate attribution over 128 users, consistent with the reliable capacity afforded by $L = 8$ under hierarchical decoding. User identities are specified via a simple CSV file defining only the number of users; no database of user-specific codewords or group assignments is stored, as both schemes derive user and group allocations dynamically from a single master key during embedding. The embedding procedure is identical across schemes, which differ only in how decoded identities are structured at attribution time. Details on parameter selection, example generations, and attribution capacity are provided in Appendix I3, Appendix I4, and Appendix H.

Each trial consists of generating one or more watermarked responses for a given prompt and user (or set of users in collusion experiments), followed by detection and attribution. In non-collusion settings, attribution returns at most one user identity, while in collusion experiments the tracer may return a bounded suspect set, as described in Section VI-D. Results are aggregated across prompts and users, with fixed random seeds used throughout for reproducibility.

B. Evaluation Metrics

Our evaluation metrics follow standard practice in recent multi-bit watermarking and source attribution studies for LLMs [17], [47], [26], [48], [46], [49], [50], [13], [27], and are used for both end-to-end attribution in the main experiments and auxiliary analyses such as bit-budget selection and parameter sweeps (Appendix I).

Attribution accuracy (non-collusion). For non-collusion attribution experiments, we report *full identity accuracy* [26], defined as the fraction of evaluated instances for which the decoded user identity matches the true generating user. This metric is used consistently for clean-text attribution and robustness experiments under single-user settings. A finer-grained analysis of hierarchical attribution behavior is provided separately in Appendix I5.

Bit-level recovery. We measure bit-level detection using *bit recovery accuracy*, defined per prompt as the fraction of correctly recovered watermark bits, following prior work [17], [48], [46]. Bits marked undecided due to low confidence are treated as incorrect. We also report the *exact match rate*, the fraction of prompts for which all L bits are recovered, and the average number of undecided bits per prompt [17].

Collusion attribution metrics. In collusion experiments, attribution is evaluated as a *suspect-set tracing* task. We report tracing accuracy as the fraction of prompts for which the tracer returns a non-empty suspect set after merging the recovered watermark signals from all colluders. To assess framing risk, we additionally report the average number of wrongly accused non-colluding users per prompt. These metrics capture the ability to localize colluding users while explicitly controlling false accusations, and are reported only for collusion settings (Section VI-D).

Framing attack metrics. To evaluate resistance to targeted framing (Section VI-D), we report the *Codeword Recovery Rate* (CRR), defined as the percentage of bits in the adversary's codeword that match an innocent victim's actual codeword. The metric quantifies how closely a coalition can reconstruct a victim's identity and is reported as the average CRR across k independent collusion trials, where k controls the number of adversarial reconstruction attempts. Unlike the Collusion attribution metrics above, CRR is computed at the codeword level rather than the suspect-set level.

Error rates. In non-collusion settings, we report *false positive* and *false negative* rates following standard definitions [47], [49], [50], [13]. False positives denote incorrect attributions, while false negatives denote missed attributions. False negative rates are below 1% across all configurations and are therefore not emphasized.

Computational metrics. In addition to attribution metrics, we evaluate system-level efficiency, including runtime and auxiliary storage overhead, reported separately in §VI-F, following prior system-level evaluations of watermarking methods [27]. All reported metrics are obtained by averaging the corresponding per-prompt values over all evaluated instances (prompts and, where applicable, collusion instances) for each experimental configuration.

C. Clean-Text Detection Performance

We first evaluate watermark detection and attribution on *unmodified* model outputs, establishing a clean baseline before considering attacks or collusion.

We evaluate eight configurations: one flat Multi-user baseline and seven Hi-DyPa splits with $L_g + L_u = 8$ and $L_u > 0$. For each configuration, a watermarked response is

generated for a selected user and decoded without any post-processing. Metrics are computed per prompt and aggregated across prompts and users.

TABLE II: Clean-text detection results. Acc. denotes full identity (user) accuracy; FP denotes false positive rate.

Scheme	(L_g, L_u)	GPT-2		DeepSeek-7B	
		Acc.	FP	Acc.	FP
MAU [8]	(0,8)	0.900	0.090	0.810	0.153
Hi-DyPa	(1,7)	0.893	0.047	0.793	0.077
Hi-DyPa	(2,6)	0.883	0.060	0.817	0.100
Hi-DyPa	(3,5)	0.897	0.047	0.853	0.093
Hi-DyPa	(4,4)	0.910	0.057	0.827	0.090
Hi-DyPa	(5,3)	0.897	0.060	0.830	0.093
Hi-DyPa	(6,2)	**0.913**	**0.030**	**0.857**	0.067
Hi-DyPa	(7,1)	0.880	0.037	0.823	**0.040**

Table II summarizes clean-text attribution performance across all configurations on both models. On unmodified outputs, Hi-DyPa achieves full identity accuracy comparable to the flat baseline across most (L_g, L_u) splits, with the strongest performance observed for intermediate partitions that balance group and user bits (e.g., $(4, 4)$ to $(6, 2)$), indicating a favorable trade-off between reliable group resolution and expressive user identification. At the same time, Hi-DyPa consistently yields lower false positive rates than flat decoding due to its hierarchical attribution procedure: user-level attribution is only attempted after a group is confidently identified, and if no group is accused, no user accusation is made. This gating mechanism reduces over-attribution while preserving clean-text detection performance, establishing a stable baseline for subsequent robustness and collusion evaluations.

D. Collusion and Framing Attacks Resistance

We can only evaluate our proposed scheme and the MAU scheme [8] on collusion and framing attacks because these schemes address the multi-user colluding scenarios. We evaluate collusion resistance in a multi-user adversarial setting, where multiple users combine independently watermarked outputs into a single response to evade attribution. We consider *same-group*, *cross-group*, and (for three-user attacks) *mixed* collusion scenarios, and aggregate results over all valid cases for each configuration.

For each (L_g, L_u) configuration, we evaluate collusions of **two** and **three** users. Following the implementation, a collusion instance is considered *successful* if the tracing procedure returns a non-empty suspect set after merging the recovered watermark signals from all colluders. Attribution under collusion is evaluated as a *suspect-set tracing* task, reflecting that the ground truth is a set of users and that combining independently generated texts dilutes the watermark signal. In this regime, suspect-set tracing provides a more reliable objective that explicitly limits false accusations.

Table III and IV summarise collusion tracing performance across all configurations and both models. For all Hi-DyPa splits, tracing success exceeds 99% for both 2- and 3-user attacks and remains stable across different (L_g, L_u) allocations, indicating strong robustness to signal averaging under collusion within the empirically supported regime $L = 8$.

TABLE III: Collusion resistance against 2-colluder attacks. Acc. denotes tracing success rate. Wrong Acc. is the average number of wrongly accused non-colluding users per prompt.

Scheme	(L_g, L_u)	GPT-2		DeepSeek-7B	
		Acc.	Wrong Acc.	Acc.	Wrong Acc.
MAU [8]	(0,8)	99.83	28.04	99.33	29.05
Hi-DyPa	(1,7)	99.00	**0.73**	94.67	**0.99**
Hi-DyPa	(2,6)	99.50	**0.73**	99.17	1.05
Hi-DyPa	(3,5)	99.83	1.39	99.50	1.41
Hi-DyPa	(4,4)	99.33	1.55	99.50	1.50
Hi-DyPa	(5,3)	**100.00**	2.22	**99.83**	1.59
Hi-DyPa	(6,2)	99.83	2.67	**99.83**	1.55
Hi-DyPa	(7,1)	99.50	3.03	**99.83**	1.61

TABLE IV: Collusion resistance against 3-colluder attacks. Acc. denotes tracing success rate. Wrong Acc. is the average number of wrongly accused non-colluding users per prompt.

Scheme	(L_g, L_u)	GPT-2		DeepSeek-7B	
		Acc.	Wrong Acc.	Acc.	Wrong Acc.
MAU [8]	(0,8)	**100.00**	26.96	**100.00**	27.33
Hi-DyPa	(1,7)	99.56	**0.44**	**100.00**	**0.69**
Hi-DyPa	(2,6)	99.89	0.46	**100.00**	0.74
Hi-DyPa	(3,5)	99.89	0.45	**100.00**	1.18
Hi-DyPa	(4,4)	**100.00**	0.55	**100.00**	1.18
Hi-DyPa	(5,3)	**100.00**	0.72	**100.00**	1.26
Hi-DyPa	(6,2)	**100.00**	0.86	**100.00**	1.42
Hi-DyPa	(7,1)	**100.00**	1.01	99.83	2.20

While the baseline achieves similarly high tracing rates, it does so by accusing many innocent users, averaging over 26 false accusations per prompt. In contrast, Hi-DyPa consistently returns small suspect sets by constraining attribution through hierarchical decoding and distance-based filtering, reducing false accusations by more than an order of magnitude across all configurations. This shows that Hi-DyPa preserves collusion traceability while substantially reducing framing risk.

We measure framing attack success using the Codeword Recovery Rate (CRR), which quantifies the percentage of bits in the adversary's codeword that match the victim's actual codeword. A CRR near 100% indicate near-complete reconstruction of the victim's codeword.

TABLE V: Framing attack resistance at $k = 50$ trials. CRR denotes Codeword Recovery Rate (%).

Scheme	(L_g, L_u)	CRR (%)	
		GPT-2	DeepSeek-7B
MAU [8]	(0,8)	79.8	91.7
Hi-DyPa	(1,7)	54.5	58.3
Hi-DyPa	(2,6)	51.0	37.5
Hi-DyPa	(3,5)	**45.5**	41.7
Hi-DyPa	(4,4)	46.5	45.8
Hi-DyPa	(5,3)	48.0	**33.3**
Hi-DyPa	(6,2)	52.0	54.2
Hi-DyPa	(7,1)	46.5	54.2

Table V reports CRR at $k = 50$ independent collusion trials on both GPT-2 and DeepSeek-7B. The flat MAU baseline is highly vulnerable to this attack on both models: adversaries reconstruct 79.8% of the victim's codeword bits on GPT-2 and 91.7% on DeepSeek-7B. The vulnerability is more severe on DeepSeek-7B because the larger model produces a higher-fidelity watermark signal, which the adversary can more reliably exploit when selecting from the feasible codeword set D1. In contrast, all Hi-DyPa configurations remain the CRR ranges from 45.5% to 54.5% on GPT-2 and from 33.3%

to 58.3% on DeepSeek-7B, with no configuration exceeding 60%. Because the dynamic codeword generation in Hi-DyPa makes the victim's codeword computationally inaccessible without the group and local seed, and forces the adversary to select essentially at random from the feasible set rather than towards the victim's codeword.

For more evaluation settings on different k colluding trials and their results, see Appendix I2.

E. Robustness Under Text Transformations

We evaluate robustness under text transformations that alter surface form while largely preserving semantics. We consider four classes of transformations: token deletion, model-based paraphrasing, synonym substitution, and LLM-based rewriting. These transformations are applied with multiple perturbation intensities and positional modes, including start, middle, end, and random. Perturbation ratios indicate the fraction of tokens or spans affected by the transformation, depending on the attack type. For brevity, we report results averaged over positional modes for each perturbation ratio. For each transformation, detection and attribution performance are evaluated using the metrics defined in §VI-B.

1) Robustness to Deletion Attacks: We first evaluate robustness to *token deletion*, which removes a fraction of tokens from the generated response and directly reduces the number of watermark-carrying positions available for detection. Token deletion is a widely used perturbation in robustness studies of LLM watermarking [15], [19], as it represents a simple yet impactful manipulation that reduces signal coverage and has been included among representative removal attacks in recent benchmark evaluations. We apply deletion to watermarked outputs and run the same detection and decoding pipeline under $L = 8$ for the baseline and all Hi-DyPa (L_g, L_u) splits with $L_g + L_u = 8$ and $L_u > 0$.

TABLE VI: Robustness to token deletion (GPT-2). Acc. denotes full identity (user) accuracy; FP denotes false positive rate.

Scheme	(L_g, L_u)	Deletion Ratio							
		0.05		0.10		0.15		0.20	
		Acc.	FP	Acc.	FP	Acc.	FP	Acc.	FP
MAU [8]	(0,8)	0.844	0.125	0.706	0.248	0.656	0.241	0.638	0.145
Hi-DyPa	(1,7)	0.768	0.106	0.640	0.131	0.609	0.089	0.593	0.071
Hi-DyPa	(2,6)	0.858	0.108	0.693	0.162	0.649	0.098	0.631	0.068
Hi-DyPa	(3,5)	0.820	0.128	0.707	0.189	0.648	0.118	0.631	0.083
Hi-DyPa	(4,4)	0.849	0.090	0.719	0.194	0.669	0.102	0.649	0.062
Hi-DyPa	(5,3)	0.820	0.099	0.688	0.177	0.639	0.101	0.621	0.071
Hi-DyPa	(6,2)	**0.877**	0.070	**0.741**	0.148	**0.683**	0.082	**0.670**	**0.043**
Hi-DyPa	(7,1)	0.839	**0.052**	0.698	**0.113**	0.647	**0.063**	0.635	0.044

Tables VI and VII summarizes full identity accuracy and false positive rates under increasing token deletion on GPT-2 and DeepSeek-7B, respectively. As deletion intensity increases, attribution accuracy decreases across all schemes due to the loss of watermark-carrying positions. The accuracy is lower on DeepSeek-7B than on GPT-2 across all deletion ratios, reflecting the larger model's more sparsely distributed entropy and fewer effective embedding positions per generation. At moderate to high deletion levels, several Hi-DyPa configurations maintain comparable or higher full identity accuracy than the baseline, reflecting the ability of hierarchical decoding to mitigate error amplification during attribution under noisy bit recovery. At the same time, Hi-DyPa consistently yields lower false positive rates across all deletion ratios, as user-level attribution is only attempted after successful group identification. Overall, these results indicate that while deletion degrades the underlying watermark signal for all methods, hierarchical decoding degrades more gracefully at the attribution stage and reduces the risk of incorrect user accusations compared to flat decoding.

TABLE VII: Robustness to token deletion (DeepSeek 7B). Acc. denotes full identity (user) accuracy; FP denotes false positive rate.

Scheme	(L_g, L_u)	Deletion Ratio							
		0.05		0.10		0.15		0.20	
		Acc.	FP	Acc.	FP	Acc.	FP	Acc.	FP
MAU [8]	(0,8)	0.581	0.354	0.498	0.391	0.451	0.355	0.412	0.315
Hi-DyPa	(1,7)	0.511	0.238	0.432	0.216	0.395	0.180	0.369	0.187
Hi-DyPa	(2,6)	0.602	0.266	0.512	0.262	0.465	0.223	0.432	0.228
Hi-DyPa	(3,5)	0.587	0.269	0.502	0.246	0.462	0.212	0.425	0.218
Hi-DyPa	(4,4)	0.612	0.280	0.514	0.266	0.476	0.227	0.442	0.234
Hi-DyPa	(5,3)	0.628	0.268	0.538	0.252	0.502	0.193	0.453	0.221
Hi-DyPa	(6,2)	**0.637**	0.203	**0.543**	0.185	**0.501**	0.161	**0.461**	0.165
Hi-DyPa	(7,1)	0.658	**0.098**	0.557	**0.101**	0.519	**0.073**	0.488	**0.088**

2) Robustness to Paraphrasing Attacks: We evaluate robustness under *model-based paraphrasing*, which rewrites text while preserving semantic content. Following recent watermark robustness studies, we implement paraphrasing attacks using a pretrained **T5** text-to-text model, a standard choice for semantic-preserving rewrites that substantially modify surface form and has been shown to effectively degrade watermark signals while maintaining fluency and meaning [20], [15], [21].

TABLE VIII: Robustness to paraphrasing (GPT-2). Acc. denotes full identity (user) accuracy; FP denotes false positive rate.

Scheme	(L_g, L_u)	Paraphrasing Ratio							
		0.05		0.10		0.15		0.20	
		Acc.	FP	Acc.	FP	Acc.	FP	Acc.	FP
MAU [8]	(0,8)	0.785	0.163	0.790	0.161	0.785	0.163	0.778	0.170
Hi-DyPa	(1,7)	0.803	0.078	0.808	0.071	0.806	0.074	0.791	0.083
Hi-DyPa	(2,6)	0.818	0.091	0.795	0.094	0.793	0.096	0.784	0.100
Hi-DyPa	(3,5)	0.823	0.088	0.817	0.095	0.823	0.092	0.820	0.092
Hi-DyPa	(4,4)	0.828	0.091	0.825	0.093	0.828	0.091	0.824	0.096
Hi-DyPa	(5,3)	**0.877**	0.063	**0.872**	0.068	**0.874**	0.065	**0.867**	0.072
Hi-DyPa	(6,2)	0.820	0.059	0.818	0.060	0.818	0.062	0.813	0.063
Hi-DyPa	(7,1)	0.836	**0.034**	0.835	**0.035**	0.844	**0.033**	0.833	**0.035**

TABLE IX: Robustness to paraphrasing (Deepseek 7B). Acc. denotes full identity (user) accuracy; FP denotes false positive rate.

Scheme	(L_g, L_u)	Paraphrasing Ratio							
		0.05		0.10		0.15		0.20	
		Acc.	FP	Acc.	FP	Acc.	FP	Acc.	FP
MAU [8]	(0,8)	0.323	0.506	0.326	0.506	0.318	0.511	0.302	0.522
Hi-DyPa	(1,7)	0.272	0.252	0.269	0.257	0.262	0.259	0.244	0.263
Hi-DyPa	(2,6)	0.342	0.326	0.342	0.324	0.347	0.315	0.321	0.335
Hi-DyPa	(3,5)	0.404	0.308	0.412	0.297	0.393	0.313	0.378	0.329
Hi-DyPa	(4,4)	**0.406**	0.322	0.398	0.327	0.393	0.318	0.379	0.327
Hi-DyPa	(5,3)	0.395	0.327	0.401	0.325	0.388	0.326	0.363	0.338
Hi-DyPa	(6,2)	0.403	0.249	**0.414**	0.242	**0.406**	0.238	**0.382**	0.260
Hi-DyPa	(7,1)	0.378	**0.164**	0.378	**0.163**	0.375	**0.152**	0.352	**0.150**

Tables VIII and IX summarize attribution performance under model-based paraphrasing on GPT-2 and DeepSeek-7B. The two models exhibit different sensitivities to T5-based paraphrasing. On GPT-2, paraphrasing degrades attribution

gracefully: full identity accuracy remains relatively stable across paraphrasing ratios, with all Hi-DyPa configurations achieving higher accuracy than the baseline. On DeepSeek-7B, paraphrasing is substantially more destructive: the flat baseline collapses to roughly 32% full identity accuracy with false positive rates above 50%, and Hi-DyPa accuracy also degrades across configurations. We attribute this gap to the interaction between the T5 paraphraser and the larger model's token distribution, which more aggressively disrupts entropy-gated embedding positions on DeepSeek-7B than on GPT-2. Despite the lower accuracy on DeepSeek-7B, the performance of Hi-DyPa over the baseline is consistent across both models: Hi-DyPa yields substantially lower false positive rates than flat decoding for the same reason observed under deletion attacks, that user attribution is only attempted after successful group identification.

3) Robustness to Synonym Substitution: We evaluate robustness under *synonym substitution*, which performs localized lexical replacements while largely preserving sentence structure and semantics. Following prior watermark robustness evaluations, we implement synonym substitution using **WordNet**-based synonym lookup, a standard approach for generating meaning-preserving lexical perturbations that alter surface form with minimal syntactic disruption [15], [23], [22], [25], [24].

TABLE X: Robustness to synonym substitution (GPT-2). Acc. denotes full identity (user) accuracy; FP denotes false positive rate.

Scheme	(L_g, L_u)	**Substitution Ratio**							
		0.05		0.10		0.15		0.20	
		Acc.	FP	Acc.	FP	Acc.	FP	Acc.	FP
MAU [8]	(0,8)	0.867	0.109	0.825	0.148	0.767	0.297	0.668	0.297
Hi-DyPa	(1,7)	0.820	0.064	0.767	0.098	0.693	0.198	0.601	0.198
Hi-DyPa	(2,6)	0.863	0.068	0.827	0.098	0.758	0.198	0.678	0.198
Hi-DyPa	(3,5)	0.865	0.088	0.817	0.116	0.777	0.223	0.688	0.223
Hi-DyPa	(4,4)	**0.881**	0.068	**0.860**	0.085	**0.818**	0.165	**0.755**	0.166
Hi-DyPa	(5,3)	0.829	0.101	0.803	0.115	0.747	0.193	0.698	0.193
Hi-DyPa	(6,2)	0.840	0.078	0.812	0.093	0.764	0.138	0.722	0.138
Hi-DyPa	(7,1)	0.853	**0.028**	0.825	**0.032**	0.783	**0.083**	0.717	**0.083**

TABLE XI: Robustness to synonym substitution (DeepSeek 7B). Acc. denotes full identity (user) accuracy; FP denotes false positive rate.

Scheme	(L_g, L_u)	**Substitution Ratio**							
		0.05		0.10		0.15		0.20	
		Acc.	FP	Acc.	FP	Acc.	FP	Acc.	FP
MAU [8]	(0,8)	0.782	0.116	0.741	0.155	0.694	0.304	0.653	0.307
Hi-DyPa	(1,7)	0.743	0.070	0.702	0.105	0.658	0.205	0.620	0.205
Hi-DyPa	(2,6)	0.785	0.075	0.744	0.105	0.721	0.205	0.685	0.205
Hi-DyPa	(3,5)	0.792	0.095	0.751	0.125	0.738	0.230	0.701	0.235
Hi-DyPa	(4,4)	**0.813**	0.075	**0.772**	0.092	**0.756**	0.172	**0.719**	0.173
Hi-DyPa	(5,3)	0.758	0.108	0.731	0.122	0.714	0.200	0.683	0.200
Hi-DyPa	(6,2)	0.774	0.085	0.748	0.100	0.729	0.145	0.697	0.145
Hi-DyPa	(7,1)	0.786	**0.032**	0.762	**0.037**	0.745	**0.090**	0.712	**0.090**

Tables X and XI summarize attribution performance under synonym substitution on GPT-2 and DeepSeek-7B, respectively. Compared to paraphrasing, synonym substitution causes a larger decrease in full identity accuracy as the substitution ratio increases, since it replaces individual words at fixed positions and directly corrupts watermark-carrying tokens, rather than redistributing content across the text. Because these replacements do not introduce new phrasing elsewhere to offset the lost signal, attribution accuracy decreases gradually with increasing substitution intensity across all schemes. Despite this stronger form of perturbation, most Hi-DyPa configurations achieve comparable or higher full identity accuracy than the baseline on both models, particularly for intermediate (L_g, L_u) splits that balance group and user resolution. At the same time, Hi-DyPa consistently yields lower false positive rates than flat decoding, reflecting the same hierarchical gating behavior observed under previous attacks, where user-level attribution is only attempted after successful group identification.

4) Robustness to LLM-based Rewriting: We evaluate robustness under *LLM-based rewriting*, which regenerates the text using a language model while attempting to preserve the original meaning. Following recent robustness evaluations of LLM watermarking, we implement rewriting attacks by prompting the same base model to rewrite its own watermarked outputs, representing a strong semantic-preserving transformation that substantially alters surface form and token distribution [20], [15].

TABLE XII: Robustness to LLM-based rewriting (GPT-2). Acc. denotes full identity (user) accuracy; FP denotes false positive rate.

Scheme	(L_g, L_u)	**Rewrite Ratio**							
		0.05		0.10		0.15		0.20	
		Acc.	FP	Acc.	FP	Acc.	FP	Acc.	FP
MAU [8]	(0,8)	0.327	0.616	0.320	0.627	0.320	0.631	0.303	0.644
Hi-DyPa	(1,7)	0.233	0.366	0.238	0.367	0.233	0.375	0.231	0.362
Hi-DyPa	(2,6)	0.317	0.447	0.325	0.440	0.322	0.438	0.304	0.450
Hi-DyPa	(3,5)	0.363	0.451	0.359	0.452	0.358	0.454	0.353	0.455
Hi-DyPa	(4,4)	0.360	0.482	0.369	0.482	0.359	0.492	0.341	0.508
Hi-DyPa	(5,3)	**0.427**	0.426	**0.411**	0.447	**0.418**	0.438	**0.403**	0.438
Hi-DyPa	(6,2)	0.369	0.441	0.372	0.407	0.380	0.395	0.367	0.408
Hi-DyPa	(7,1)	0.359	**0.186**	0.363	**0.190**	0.361	**0.198**	0.355	**0.197**

TABLE XIII: Robustness to LLM-based rewriting (DeepSeek 7B). Acc. denotes full identity (user) accuracy; FP denotes false positive rate.

Scheme	(L_g, L_u)	**Rewrite Ratio**							
		0.05		0.10		0.15		0.20	
		Acc.	FP	Acc.	FP	Acc.	FP	Acc.	FP
MAU [8]	(0,8)	0.305	0.638	0.298	0.645	0.291	0.652	0.279	0.661
Hi-DyPa	(1,7)	0.215	0.382	0.219	0.385	0.211	0.391	0.208	0.388
Hi-DyPa	(2,6)	0.294	0.466	0.301	0.461	0.297	0.459	0.281	0.471
Hi-DyPa	(3,5)	0.339	0.470	0.334	0.473	0.331	0.475	0.325	0.479
Hi-DyPa	(4,4)	0.336	0.503	0.342	0.505	0.333	0.514	0.318	0.529
Hi-DyPa	(5,3)	**0.402**	0.448	**0.387**	0.467	**0.394**	0.459	**0.379**	0.461
Hi-DyPa	(6,2)	0.346	0.462	0.349	0.428	0.355	0.418	0.342	0.430
Hi-DyPa	(7,1)	0.336	**0.205**	0.341	**0.211**	0.338	**0.219**	0.331	**0.220**

Tables XII and XIII summarize attribution performance under LLM-based rewriting on GPT-2 and DeepSeek-7B. Rewriting is the most destructive transformation, as regenerating the text largely decorrelates the output from the original token-level sampling process, leading to severely degraded bit recovery and low full identity accuracy across all schemes on both models. In this high-uncertainty regime, flat decoding frequently converts near-random recovered bits into confident but incorrect user accusations, resulting in extremely high false positive rates. In contrast, Hi-DyPa yields dramatically lower false positives by limiting how uncertainty propagates during attribution: group-level decoding often fails early under rewriting, preventing downstream user attribution, and even

when a group is selected, user decoding is restricted to a smaller candidate set, reducing the likelihood of spurious user accusations. As a result, Hi-DyPa remains substantially more conservative and reliable than flat decoding under this strongest attack, despite the overall collapse in attribution accuracy.

F. Computational Overhead/ Efficiency

We measure the per-run computational overhead of watermarking by reporting initialization cost, embedding time, detection time, tracing time, and additional metadata storage. Our evaluation of time and memory overhead follows the methodology used in recent system-level studies of LLM watermarking and attribution, which report runtime and auxiliary storage costs to assess practical deployability [27].

Table XIV shows that Hi-DyPa achieves substantially faster tracing times, consistently 1.5–2$\times$ lower than the MAU baseline, because hierarchical decoding first identifies a candidate group and then traces only within that group, rather than evaluating all users.

TABLE XIV: Tracing time (milliseconds) per run across configurations on GPT-2 and DeepSeek-7B models.

Scheme	(L_g, L_u)	**GPT-2 Trace (ms)**	**DeepSeek-7B Trace (ms)**
MAU [8]	(0,8)	0.429	4.23
Hi-DyPa	(1,7)	0.556	18.06
Hi-DyPa	(2,6)	0.350	10.76
Hi-DyPa	(3,5)	0.249	6.12
Hi-DyPa	(4,4)	0.216	3.51
Hi-DyPa	(5,3)	**0.197**	2.57
Hi-DyPa	(6,2)	0.209	2.06
Hi-DyPa	(7,1)	0.227	**1.46**

In table XV, we compare the tracing time of our scheme running on the Deepseek-7B model at an 8-bit codeword length to existing watermarking schemes, including: Fernandez et al. [45], Wang et al. [40], Yoo et al [46], Multi-bit scheme [17], and MAU [8] at a 12-bit message length running on the same model scale as ours. We also compare the tracing time of our scheme to MAU scheme for 8-bit message. We choose our best configuration result as $(7, 1)$ setting The reason for this bit length choice is the trade-off between the embedded message length and the tracing time. The longer the watermark bit length, the longer the tracing time [17]. Moreover, our scheme has been proven robust even with a short fixed bit length (VI-E1, VI-E4, VI-E2, VI-E3, VI-E4, and VI-D), ensuring its efficiency. The efficiency of our scheme is guaranteed by the hierarchy of the system and the chosen set of users and groups.

As shown in table XV, even at the same bit length as our scheme, the MAU [8] scheme's tracing time is slower than our scheme.

Additionally, we include the full metadata storage, system initialization time, embedding time and detecting time experiment in tables XIX and XX in Appendix I1.

VII. Discussion

Hi-DyPa provides more reliable attribution than flat decoding by controlling how uncertainty in watermark recovery propagates during attribution. Because hierarchical decoding enforces a group-first, user-later gating structure, user-level attribution is only attempted when group identification succeeds. This substantially reduces false positives and false accusations across clean-text, collusion, and attack settings, while maintaining comparable full-identity accuracy to the baseline. Under collusion, Hi-DyPa preserves high tracing success while returning much smaller suspect sets, reducing framing risk by more than an order of magnitude. The same gating effect improves robustness under text transformations and LLM-based rewriting, where flat decoding often converts noisy signals into confident but incorrect accusations. In addition, hierarchical tracing improves efficiency: by restricting user-level tracing to a single group, Hi-DyPa achieves significantly lower tracing time than flat decoding, while incurring comparable embedding and detection costs.

TABLE XV: Tracing time comparison across watermarking schemes. All evaluated on 7B-parameter models.

Scheme	**Bits**	**Trace (ms)**
Fernandez et al. [45]	12	120
Wang et al. [40]	12	160
Yoo et al. [46]	12	10
Multi-bit [17]	12	40
MAU [8]	12	10
MAU [8]	8	4.23
Hi-DyPa	8	**1.46**

Several related aspects are deferred to the appendix, including graceful degradation via group-only or abstained attribution when user-level evidence is weak (Appendix H3, Appendix I5), as well as memory overhead, system-level trade-offs, and scaling behavior under larger models or longer generations (Appendix J).

Our scheme mainly focuses on increasing the layers rather than the bit size. However, extended work to increase the bit length but still preserve the *Robustness* and *Efficiency* can be considered in the future. More details of future directions for our scheme will be discussed in Appendix J.

VIII. Conclusion

We present Hi-DyPa, a practical hierarchical dynamic parallel watermarking framework for multi-user LLM attribution that addresses key limitations of prior schemes, including scalability bottlenecks, efficiency degradation, and framing risks from shared static codewords. By combining hierarchical identity decoding with PRF-based dynamic local codewords and parallel embedding, Hi-DyPa achieves robust attribution under text modification, collusion, and framing while inheriting the core security guarantees of the underlying watermarking framework. Extensive experiments demonstrate strong clean-text attribution, high tracing success under multi-user collusion, and consistently lower false accusation rates than flat baselines, even under stronger text transformations. Future work includes evaluation on larger modern LLMs, exploring how longer generations and improved recovery expand the effective bit budget, and formalizing partial attribution as a first-class system output.

Ethical Consideration

We use publicly accessible datasets and the GPT-2 and Deepseek-7B model to investigate Hi-DyPa watermarking design for LLM systems. The datasets do not contain sensitive or identifiable information. The experimental and theoretical evaluation does not pose any safety risks or concerns, nor does it involve harm to any commercial enterprises or products on the market. To the best of our knowledge, there are no concerns related to human ethics. We do not make any comparisons with real-world industrial products. Hence, there is no implicit impact on any company or organization. Furthermore, we use public and standard benchmark metrics in machine learning without involving any social or human-centric security studies.

Open Science Policy

Hi-DyPa is open source and available at https://anonymous.4open.science/r/hidypa-1ED8/. The implementation contains existing multi-user watermarking schemes [8], and our baseline schemes and Hi-DyPa scheme.

References

[1] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877–1901, 2020.

[2] R. Zellers, A. Holtzman, H. Rashkin, Y. Bisk, A. Farhadi, F. Roesner, and Y. Choi, "Defending against neural fake news," *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[3] G. Jawahar, M. Abdul-Mageed, and L. V. S. Lakshmanan, "Automatic detection of machine generated text: A critical survey," in *Proceedings of the 28th International Conference on Computational Linguistics (COLING)*, 2020, pp. 2296–2309.

[4] H. Chen, C. Liu, T. Zhu, and W. Zhou, "When deep learning meets watermarking: A survey of application, attacks and defenses," *Computer Standards & Interfaces*, vol. 89, p. 103830, 2024.

[5] Google DeepMind, "SynthID," 2023, accessed: 2026-01-09. [Online]. Available: https://deepmind.google/models/synthid/

[6] P.-L. Lin, "Digital watermarking models for resolving rightful ownership and authenticating legitimate customer," *Journal of Systems and Software*, vol. 55, no. 3, pp. 261–271, 2001.

[7] Z. Jiang, M. Guo, Y. Hu, Y. Wang, and N. Z. Gong, "Watermark-based attribution of AI-generated content," 2024.

[8] A. Cohen, A. Hoover, and G. Schoenbach, "Watermarking language models for many adaptive users," in *Proceedings of the 2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2025, pp. 2583–2601.

[9] J. Kirchenbauer, J. Geiping, Y. Wen, J. Katz, I. Miers, and T. Goldstein, "A watermark for large language models," in *Proceedings of the 40th International Conference on Machine Learning (ICML)*. PMLR, 2023, pp. 17 061–17 084.

[10] A. Liu, L. Pan, X. Hu, S. Meng, and L. Wen, "A semantic invariant robust watermark for large language models," in *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024.

[11] S. Dathathri, A. See, S. Ghaisas, P.-S. Huang, R. McAdam, J. Welbl, V. Bachani, A. Kaskasoli, R. Stanforth, T. Matejovicova *et al.*, "Scalable watermarking for identifying large language model outputs," *Nature*, vol. 634, no. 8035, pp. 818–823, 2024.

[12] A. Liu, L. Pan, Y. Lu, J. Li, X. Hu, X. Zhang, L. Wen, I. King, H. Xiong, and P. Yu, "A survey of text watermarking in the era of large language models," *ACM Computing Surveys*, vol. 57, no. 2, pp. 1–36, 2024.

[13] J. Kirchenbauer, J. Geiping, Y. Wen, M. Shu, K. Saifullah, K. Kong, K. Fernando, A. Saha, M. Goldblum, and T. Goldstein, "On the reliability of watermarks for large language models," in *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024.

[14] X. Zhao, S. Gunn, M. Christ, J. Fairoze, A. Fabrega, N. Carlini, S. Garg, S. Hong, M. Nasr, F. Tramer, S. Jha, L. Li, Y.-X. Wang, and D. Song, "SoK: Watermarking for AI-generated content," in *Proceedings of the 2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2025, pp. 2621–2639.

[15] J. Liang, Z. Wang, S. Hong, S. Ji, and T. Wang, "Watermark under fire: A robustness evaluation of LLM watermarking," in *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025, pp. 21 050–21 074.

[16] W. Zhong, X. Huang, and X. Zhang, "Quantitative frameproof codes and hypergraphs," 2025.

[17] W. Qu, W. Zheng, T. Tao, D. Yin, Y. Jiang, Z. Tian, W. Zou, J. Jia, and J. Zhang, "Provably robust multi-bit watermarking for AI-generated text," in *Proceedings of the 34th USENIX Security Symposium*, 2025, pp. 201–220.

[18] M. Christ, S. Gunn, and O. Zamir, "Undetectable watermarks for language models," in *Proceedings of the 37th Annual Conference on Learning Theory (COLT)*. PMLR, 2024, pp. 1125–1139.

[19] N. Golowich and A. Moitra, "Edit distance robust watermarks for language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[20] Y. Cheng, H. Guo, Y. Li, and L. Sigal, "Revealing weaknesses in text watermarking through self-information rewrite attacks," in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025, pp. 9982–10 009.

[21] R. Zhang, S. S. Hussain, P. Neekhara, and F. Koushanfar, "REMARK-LLM: A robust and efficient watermarking framework for generative large language models," in *Proceedings of the 33rd USENIX Security Symposium*, 2024, pp. 1813–1830.

[22] E. German, S. Antebi, E. Habler, A. Shabtai, and Y. Elovici, "LexiMark: Robust watermarking via lexical substitutions to enhance membership verification of an LLM's textual training data," 2025.

[23] L. Pan, A. Liu, Z. He, Z. Gao, X. Zhao, Y. Lu, B. Zhou, S. Liu, X. Hu, L. Wen, I. King, and P. S. Yu, "MarkLLM: An open-source toolkit for LLM watermarking," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP)*, 2024, pp. 61–71.

[24] X. Zhao, P. Ananth, L. Li, and Y.-X. Wang, "Provable robust watermarking for AI-generated text," in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

[25] X. Zhao, Y.-X. Wang, and L. Li, "Protecting language generation models via invisible watermarking," in *Proceedings of the 40th International Conference on Machine Learning (ICML)*. PMLR, 2023, pp. 42 187–42 199.

[26] X. Lu, J. Wang, Z. Zhao, Z. Dai, C.-S. Foo, S. K. Ng, and B. K. H. Low, "WASA: Watermark-based source attribution for large language model-generated data," in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025, pp. 23 791–23 824.

[27] S. Guo, Y. Ju, X. Chen, and J. Gu, "LLM-MARK: A computing framework on efficient watermarking of large language models for authentic use of generative AI at local devices," in *Proceedings of the 61st ACM/IEEE Design Automation Conference (DAC)*, 2024, pp. 1–6.

[28] D. Boneh and J. Shaw, "Collusion-secure fingerprinting for digital data," *IEEE Transactions on Information Theory*, vol. 44, no. 5, pp. 1897–1905, 1998.

[29] D. Boneh, A. Kiayias, and H. W. Montgomery, "Robust fingerprinting codes: A near optimal construction," in *Proceedings of the 10th ACM Workshop on Digital Rights Management (DRM)*, Chicago, IL, USA, October 2010, pp. 3–12.

[30] G. Tardos, "Optimal probabilistic fingerprint codes," *Journal of the ACM*, vol. 55, no. 2, pp. 1–24, 2008.

[31] D. Comer, "Ubiquitous B-Tree," *ACM Computing Surveys*, vol. 11, no. 2, pp. 121–137, 1979.

[32] M. Cheng and Y. Miao, "On anti-collusion codes and detection algorithms for multimedia fingerprinting," *IEEE Transactions on Information Theory*, vol. 57, no. 7, pp. 4843–4851, 2011.

[33] N. Lukas and F. Kerschbaum, "PTW: Pivotal tuning watermarking for pre-trained image generators," in *Proceedings of the 32nd USENIX Security Symposium*. USENIX Association, 2023, pp. 2241–2258.

[34] C. Sun and E.-H. Yang, "A watermarking-based framework for protecting deep image classifiers against adversarial attacks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, 2021, pp. 3329–3338.

[35] S. Shao, Y. Li, H. Yao, Y. He, Z. Qin, and K. Ren, "Explanation as a watermark: Towards harmless and multi-bit model ownership verification via watermarking feature attribution," in *Proceedings of the 32nd Annual Network and Distributed System Security Symposium (NDSS)*, 2025.
[36] S. Martínez, S. Gérard, and J. Cabot, "On watermarking for collaborative model-driven engineering," *IEEE Access*, vol. 6, pp. 29 715–29 728, 2018.
[37] J. Griffin, P. Yuan, K. R. Duffy, and M. Médard, "Using a single parity-check to reduce the guesswork of guessing codeword decoding," in *Proceedings of the 59th Annual Conference on Information Sciences and Systems (CISS)*. IEEE, 2025, pp. 1–6.
[38] A. Liu, L. Pan, X. Hu, S. Li, L. Wen, I. King, and P. S. Yu, "An unforgeable publicly verifiable watermark for large language models," in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
[39] X. Li, G. Li, and X. Zhang, "Segmenting watermarked texts from language models," *Advances in Neural Information Processing Systems*, vol. 37, pp. 14 634–14 665, 2024.
[40] L. Wang, W. Yang, D. Chen, H. Zhou, Y. Lin, F. Meng, J. Zhou, and X. Sun, "Towards codable watermarking for injecting multi-bits information to LLMs," in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
[41] A. Fan, Y. Jernite, E. Perez, D. Grangier, J. Weston, and M. Auli, "ELI5: Long form question answering," in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019, pp. 3558–3567.
[42] S. Longpre, L. Hou, T. Vu, A. Webson, H. W. Chung, Y. Tay, D. Zhou, Q. V. Le, B. Zoph, J. Wei, and A. Roberts, "The FLAN collection: Designing data and methods for effective instruction tuning," 2023.
[43] X. Y. Huang, K. Vishnubhotla, and F. Rudzicz, "The GPT-WritingPrompts dataset: A comparative analysis of character portrayal in short stories," 2024.
[44] F. M. Polo, R. Xu, L. Weber, M. Silva, O. Bhardwaj, L. Choshen, A. F. M. de Oliveira, Y. Sun, and M. Yurochkin, "Efficient multi-prompt evaluation of LLMs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
[45] P. Fernandez, A. Chaffin, K. Tit, V. Chappelier, and T. Furon, "Three bricks to consolidate watermarks for large language models," in *2023 IEEE International Workshop on Information Forensics and Security (WIFS)*, 2023, pp. 1–6.
[46] K. Yoo, W. Ahn, and N. Kwak, "Advancing beyond identification: Multi-bit watermark for large language models," in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, 2024, pp. 4031–4055.
[47] Z. Fu and C. Russell, "Multi-use LLM watermarking and the false detection problem," 2025.
[48] Z. Zhang, R. Jin, G. Xu, X. Wang, M. Zitnik, L. Cong, and M. Wang, "FoldMark: Safeguarding protein structure generative models with distributional and evolutionary watermarking," *bioRxiv*, 2025.
[49] Y. Liang, J. Xiao, W. Gan, and P. S. Yu, "Watermarking techniques for large language models: A survey," *Artificial Intelligence Review*, 2026, in press.
[50] H. He, Y. Liu, Z. Wang, Y. Mao, and Y. Bu, "Theoretically grounded framework for LLM watermarking: A distribution-adaptive approach," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
[51] B. Skoric, T. U. Vladimirova, M. Celik, and J. C. Talstra, "Tardos fingerprinting is better than we thought," *IEEE Transactions on Information Theory*, vol. 54, no. 8, pp. 3663–3676, 2008.
[52] A. Barg, G. R. Blakley, and G. A. Kabatiansky, "Digital fingerprinting codes: Problem statements, constructions, identification of traitors," *IEEE Transactions on Information Theory*, vol. 49, no. 4, pp. 852–865, 2003.
[53] D. Ostrev, D. Orsucci, F. Lázaro, and B. Matuz, "Classical product code constructions for quantum Calderbank-Shor-Steane codes," *Quantum*, vol. 8, p. 1420, 2024.
[54] Y. Liu and Y. Bu, "Adaptive text watermark for large language models," in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024, pp. 30 718–30 737.
[55] Y. Lu, A. Liu, D. Yu, J. Li, and I. King, "An entropy-based text watermarking detection method," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024, pp. 11 724–11 735.
[56] R. P. Hastuti, R. A. Rajagede, M. A. Ghanim, M. Zheng, and Q. Lou, "Factuality beyond coherence: Evaluating LLM watermarking methods for medical texts," in *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025, pp. 15 129–15 147.

APPENDIX

A. Related Existing Watermarking Schemes

Zero-bit Watermarking Scheme: Christ et al [18] propose the first cryptographic Zero-bit watermarking scheme for language models. Their scheme embeds hidden patterns in the model-generated text with high empirical entropy using the pseudorandom function PRF. The Zero-bit scheme achieves *Undetectability, Soundness, and Robustness*. To ensure *Undetectability*, the secret-key PRF is applied to implicitly define preferred tokens at each position. This negligible bias is applied to ensure that adversaries cannot distinguish the watermarked output from the unwatermarked one. For *Robustness* guarantee, the detector recomputes PRF values, and the detection is successful if there are sufficiently high-entropy segments of watermarked text still remaining, even after the text has been edited. Their *Soundness* ensures negligible false positive detection error with cryptographic proofs [18]. However, the Zero-bit scheme lacks traceability; it is binary detection without user attribution, which could not identify which LLM users generate the text.

L-bit Watermarking Scheme: Motivated by the lack of ability to trace content back to specific users of the Zero-bit scheme [18], the L-bit scheme extends the previous scheme by embedding an L-bit message into generated text [8]. To embed this message, the scheme applies the underlying zero-bit watermark L times in parallel. To extract the embedded message, the extractor runs the zero-bit detector. The extraction succeeds whenever it approximates enough blocks to detect watermarks [8]. By assigning each user a unique L-bit identifier, the scheme can extract the specific user's identification from the watermarked text. The L-bit scheme inherits the watermarking properties from the Zero-bit scheme and introduces a new property—*Completeness*: a highly correct and sufficient message is extractable from the watermarked text [8]. Although the L-bit scheme has the tracing capability for a specific user, it is vulnerable to the collusion attack. When multiple users collude by combining their model-generated outputs, the extractor cannot identify the mixture of embedded watermarked messages to any specific group of colluding users [8].

Multiple Adaptive User (MAU) Watermarking Scheme: The Multiple Adaptive User scheme [8] can trace back to the users who generated the text, even when they collude [28], [29], [30]. It replaces the user identifiers with codewords from collusion-resistant fingerprinting codes. These codes provide a mathematical guarantee that colluders could not evade tracing. The scheme is constructed by combining L-bit watermarking with these fingerprinting codes. All discussed watermarking properties of this scheme are preserved due

to its construction from the previous L-bit scheme and its provably robust and collusion-resistant fingerprinting codes.

Multi-bit Watermarking Scheme: The provably robust Multi-bit Watermarking Scheme [17] addresses the scalability limitations of existing approaches for embedding long messages into LLM-generated text. Their approach is to partition the embedded bit-messages into segments using pseudo-random assignment combined with Reed-Solomon error correction codes. Their scheme achieves high extraction accuracy while maintaining computational efficiency, which addresses the challenges in extraction efficiency or accuracy under longer embedded message length of the previous works [46], [40]. However, their scheme focuses on a single-user scenario and does not address the multi-user collusion resistance.

B. Key Watermarking Operations

Watermarking Scheme Key Components Key components that the Multi-user watermarking scheme will generate during the key generation stage [8]:

- **Master secret key**: Controls watermark embedding and detection
- **Tracing key**: Enables user identification from extracted watermarks
- **User codewords / User Identification**: Unique binary sequences for each user in the system. For example, an 8-bit identification codeword for user Bob is 01010001.
- **PRF seeds**: Enable deterministic generation of user-specific parameters

Key Generation (KeyGen): Key generation establishes the foundation for the entire watermarking system. The secret key is used both for embedding watermarks during text generation and detecting them afterwards. This is analogous to a bank creating a master key that can verify all customer signatures, while each customer receives their own unique signature pattern.

Embedding (Wat): The embedding algorithm modifies the text generation process to encode invisible signals without degrading output quality. For Zero-bit watermarking scheme [18], this simply marks text as AI-generated. For multi-user schemes [8], it additionally encodes user-identifying information through fingerprinting codewords. When a student Bob asks the LLM "Explain photosynthesis", the system retrieves Bob's unique identification codeword in the university LLM system. The system subtly favors tokens that encode Bob's bits and output: "Photosynthesis is the process by which plants convert sunlight into energy...". The output will include Bob's codeword signature, which is hidden within that token choice. When choosing between "convert" and "transform" (both semantically valid), the PRF output and Bob's codeword bit determine which token is selected [9].

Detection (Detect): Detection determines whether a given text was generated by the watermarked model. This is a binary classification that does not identify specific users; it only answers "Is this AI-generated text from our system?" For instance, the professor suspects a student submitted AI-generated work. The detection process works like checking for the existence of a hidden watermark in the text. The detector uses the secret key to reconstruct what "green" tokens would have been at each position. If significantly more green tokens appear than expected by chance, the text is flagged as watermarked.

Tracing (Trace): Tracing goes beyond detection to identify which specific users generated the text. This enables accountability in multi-user deployments. The algorithm extracts the embedded codeword and compares it against known user codewords to identify the source. Continuing with the university scenario, a leaked exam solution is found online. First, the detection confirms the text is watermarked (from our system). Then the tracer extracts the embedded bits from the text, and the system compares them against the codeword database. The user codeword that matches the most bits in the extracted bit message is identified as the source of the leaked exam solution.

C. Accusation Determination for Tracing

For each candidate user u, we compute a normalized score between the extracted local portion $\hat{m}_u$ and the candidate codeword $X_u[u]$: $\text{score}_u := 1 - \frac{\text{Hamming}(\hat{m}_u, X_u[u])}{L_u}$

Here, $\text{Hamming}(\cdot,\cdot)$ counts the number of indices $i \in [L_u]$ at which the two strings disagree. A user is accused if their similarity score exceeds a user-level threshold τ_u.

D. Collusion and Framing Attacks

1) Collusion Strategies: Let $\mathcal{C} = \{c_1, c_2, \ldots, c_n\}$ be a set of n codewords assigned to users, where each $c_i \in \{0,1\}^L$ is an L-bit fingerprinting code. For a coalition $\mathcal{T} = \{u_1, \ldots, u_c\}$ of c colluding users with codewords $\{X_{u_1}, \ldots, X_{u_c}\}$, the fingerprinting code's principle introduced by Boneh and Shaw [28] states that at any position $i \in [L]$, if all colluders have the same bit value at position i: $X_{u_1}[i] = X_{u_2}[i] = \cdots = X_{u_c}[i] = b$, the colluders cannot produce a different bit value at position i. In contrast, if colluders have different bit values at position i, they can produce any valid bit value in that position.

The feasible set consists of all possible colluding codewords that the colluding adversaries can produce. Any robust fingerprinting code must ensure that the tracing algorithm can identify at least one colluder from any codeword in $\mathcal{F}(\mathcal{T})$ [30], [51], [52].

Collusion and framing attacks can be categorized based on the colluder's information access as follows:

- **Codeword-Blind:** Colluders know only their watermarked text but not their assigned codewords or the codeword structure [30].
- **Full Codeword-Aware:** Colluders know their own codewords and can compute the feasible codeword set. This

would allow them to select the bits at specific bit positions [30]. In the context of framing attack, the adversaries knew both their own codewords and the innocent codewords for framing.

- **Semi Codeword-Aware:** The adversaries knew only their own codeword, not the innocent user's codeword.

2) Framing Attacks: Consider a system with 3 users and 6-bit codeword assignments:

TABLE XVI: User Codeword Assignments

User	Codeword	Role
Alice	100110	Colluder
Bob	010011	Colluder
Carol	110010	**Innocent**

The colluders (Alice and Bob) analyze their codewords position by position to determine the feasible bit values they can produce, as formalized in quantitative frameproof code analyses [16].

TABLE XVII: Position-by-Position Analysis of Colluders' Capabilities

Position	Alice	Bob	Can Produce
1	1	0	$\{0,1\}$
2	0	1	$\{0,1\}$
3	0	0	$\{0\}$
4	1	0	$\{0,1\}$
5	1	1	$\{1\}$
6	0	1	$\{0,1\}$

At positions 3 and 5, both colluders share the same bit value, so these positions are fixed in any feasible colluding codeword. At all other positions, the colluders have the freedom to choose either bit value. Alice and Bob can construct the colluding codeword **110010**. The constructed codeword **110010** is exactly Carol's codeword. Therefore, the tracing algorithm will accuse Carol, an innocent user. The tracing algorithm will identify the user with the minimum Hamming distance to the recovered codeword. However, this attack only occurs when the colluding adversaries have access to the codeword database or the system secret key.

E. Proposed Scheme Constructions and Algorithms

1) Basic Static Hierarchical Scheme Constructions:

Definition A.1 (Hierarchical Static Watermarking)**.** A hierarchical watermarking scheme for a Model over a token alphabet $\mathcal{T}$ and a set of users $\mathcal{U}$ organized into G groups of s users is a tuple of algorithms:
$\mathcal{W}_{\mathsf{static-hier}} = (\mathsf{KeyGen}, \mathsf{Wat}, \mathsf{Detect}, \mathsf{Trace})$, where:

- $\mathsf{KeyGen}(1^\lambda) \to \mathsf{sk}$ is a randomized algorithm that takes a security parameter λ as input and outputs a secret key sk.
- $\mathsf{Wat}_{\mathsf{sk}}(u, Q) \to T$ is a keyed randomized algorithm that takes as input a user $u \in \mathcal{U}$ and a prompt $Q \in \mathcal{T}^*$ and generates a response string $T \in \mathcal{T}^*$.
- $\mathsf{Detect}_{\mathsf{sk}}(T) \to b$ is a keyed deterministic algorithm that takes a string $T \in \mathcal{T}^*$ as input and outputs a bit $b \in \{0,1\}$.
- $\mathsf{Trace}_{\mathsf{sk}}(T) \to S$ is a keyed deterministic algorithm that takes a string $T \in \mathcal{T}^*$ as input and outputs a set of accused users $S \subseteq \mathcal{U}$.

Constructed from:

- A group-layer watermarking scheme: $\mathcal{W}_g = (\mathsf{KeyGen}_g, \mathsf{Wat}_g, \mathsf{Detect}_g, \mathsf{Trace}_g)$ as defined in Definition A.2
- A user-layer watermarking scheme: $\mathcal{W}_u = (\mathsf{KeyGen}_u, \mathsf{Wat}_u, \mathsf{Trace}_u)$ as defined in Definition A.3

Definition A.2 (Group-layer watermarking)**.** A group-layer watermarking scheme for a model Model over a token alphabet $\mathcal{T}$ and a set of groups $\mathcal{G}$ is a tuple of efficient algorithms $\mathcal{W}_g = (\mathsf{KeyGen}_g, \mathsf{Wat}_g, \mathsf{Detect}_g, \mathsf{Trace}_g)$.

Where:

- $\mathsf{KeyGen}_g(1^\lambda) \to \mathsf{sk}_g$ is a randomized algorithm that takes a security parameter λ as input and outputs a secret key sk_g.
- $\mathsf{Wat}_{g,\mathsf{sk}_g}(g, Q) \to T$ is a keyed randomized algorithm that takes as input a group $g \in [G]$ and a prompt $Q \in \mathcal{T}^*$ and generates a response string $T \in \mathcal{T}^*$.
- $\mathsf{Detect}_{g,\mathsf{sk}_g}(T) \to b$ is a keyed deterministic algorithm that takes a string $T \in \mathcal{T}^*$ as input and outputs a bit $b \in \{0,1\}$.
- $\mathsf{Trace}_{g,\mathsf{sk}_g}(T) \to S_g$ is a keyed deterministic algorithm that takes a string $T \in \mathcal{T}^*$ as input and outputs a set of accused groups $S_g \subseteq [G]$.

Constructed from:

- An multi-user watermarking scheme: $\mathcal{W}' = (\mathsf{KeyGen}', \mathsf{Wat}', \mathsf{Detect}', \mathsf{Extract}')$ for the group layer
- A robust fingerprinting code: $\mathsf{FP}_g = (\mathsf{FP.Gen}_g, \mathsf{FP.Trace}_g)$ for group identification

Definition A.3 (User-layer watermarking)**.** A user-layer watermarking scheme for a model Model over a token alphabet $\mathcal{T}$ and a set of user positions $[s]$ within each group is a tuple of efficient algorithms $\mathcal{W}_u = (\mathsf{KeyGen}_u, \mathsf{Wat}_u, \mathsf{Trace}_u)$.

Where:

- $\mathsf{KeyGen}_u(1^\lambda) \to \mathsf{sk}_u$ is a randomized algorithm that takes a security parameter λ as input and outputs a secret key sk_u.
- $\mathsf{Wat}_{u,\mathsf{sk}_u}(u, Q) \to T$ is a keyed randomized algorithm that takes as input a user position $u \in [s]$ and a prompt $Q \in \mathcal{T}^*$ and generates a response string $T \in \mathcal{T}^*$.
- $\mathsf{Trace}_{u,\mathsf{sk}_u}(\hat{m}_u) \to u^*$ is a keyed deterministic algorithm that takes an extracted user message $\hat{m}_u \in \{0,1,\perp\}^{L_u}$ as input and outputs an accused user $u^* \in [s] \cup \{\perp\}$.

Constructed from:

- A multi-user watermarking scheme: $\mathcal{W}' = (\mathsf{KeyGen}', \mathsf{Wat}', \mathsf{Extract}')$ for user-layer embed
- A fingerprinting code $\mathsf{FP}_u = (\mathsf{FP.Gen}_u, \mathsf{FP.Trace}_u)$ for user identification within groups

2) Hierarchical Static Key Generation: See Algorithm 1 for Hierarchical Static Key Generation.

Algorithm 1 Hierarchical Static Scheme: KeyGen

$\mathsf{sk} \leftarrow \mathsf{KeyGen}'(1^\lambda)$ ▷ Watermarking secret key
$(X, \mathsf{tk}) \leftarrow \mathsf{FP.Gen}(1^\lambda, G)$ ▷ Group fingerprinting codes
for $g \in [G]$ **do**
  $X_g[g] \leftarrow X[g]$ ▷ Assign group codeword
end for
for $u \in [s]$ **do**
  $X_u[u] \xleftarrow{\$} \{0,1\}^{L_u}$ ▷ Random local codeword (shared across groups)
end for
return $\{\mathsf{sk}, \mathsf{tk}, X_g, X_u\}$

3) Hierarchical Dynamic Key Generation: See Algorithm 2.

Algorithm 2 Hierarchical Dynamic Scheme: KeyGen

$\mathsf{sk} \leftarrow \mathrm{KeyGen}'(1^\lambda)$ ▷ Secret key
$(X, \mathsf{tk}) \leftarrow \mathrm{FP.Gen}(1^\lambda, G)$ ▷ Group fingerprinting codes
for $g \in [G]$ **do**
  $X_g[g] \leftarrow X[g]$ ▷ Assign group codeword
end for
for $g \in [G]$ **do**
  $K_g[g] \xleftarrow{\$} \{0,1\}^\lambda$ ▷ Secret group seed
end for
for $u \in [s]$ **do**
  $T_u[u] \xleftarrow{\$} \{0,1\}^\lambda$ ▷ Public position template
end for
return $\{\mathrm{sk}, \mathsf{tk}, X_g, K_g, T_u\}$

4) Hi-DyPa Key Generation: See Algorithm 3.

Algorithm 3 Hi-DyPa Scheme: KeyGen

$\mathsf{sk} \leftarrow \textsc{KeyGen}'(1^\lambda)$ ▷ Watermarking secret key
$(X, \mathsf{tk}) \leftarrow \mathsf{FP.Gen}(1^\lambda, G)$ ▷ Group fingerprinting codes
for $g \in [G]$ **do**
  $X_g[g] \leftarrow X[g]$ ▷ Assign group codeword
end for
for $g \in [G]$ **do**
  $K_g[g] \xleftarrow{\$} \{0,1\}^\lambda$ ▷ Secret group seed
end for
for $u \in [s]$ **do**
  $T_u[u] \xleftarrow{\$} \{0,1\}^\lambda$ ▷ Public position template
end for
return $\{\mathsf{sk}, \mathsf{tk}, X_g, K_g, T_u, r\}$

5) Hierarchical Static Embedding: See Algorithm 4.

Algorithm 6 Hi-DyPa Scheme: Parallel Codeword Embedding

▷ Step 1: Construct composite codeword (same as Hierarchical Dynamic, see algorithm 5)

▷ Step 2: Parallel random assignment of positions to layers

$\mathsf{idx}_g \leftarrow 0$; $\mathsf{idx}_u \leftarrow 0$
for $i = 1$ to m **do**
  if $\textsc{Random}() < r$ **then** ▷ With probability r
    $\mathsf{layer}[i] \leftarrow$ "group"
    $\mathsf{bit}[i] \leftarrow C_g[\mathsf{idx}_g \mod L_g]$
    $\mathsf{idx}_g \leftarrow \mathsf{idx}_g + 1$
  else ▷ With probability $(1 - r)$
    $\mathsf{layer}[i] \leftarrow$ "user"
    $\mathsf{bit}[i] \leftarrow C_u[\mathsf{idx}_u \mod L_u]$
    $\mathsf{idx}_u \leftarrow \mathsf{idx}_u + 1$
  end if
end for
return $\{\mathsf{layer}[], \mathsf{bit}[]\}$

Algorithm 4 Hierarchical Static Scheme: Wat (Embedding)

$C_g \leftarrow X_g[g]$ ▷ Retrieve group codeword
$C_u \leftarrow X_u[u]$ ▷ Retrieve local codeword
$C \leftarrow C_g \| C_u$ ▷ Composite codeword: group ‖ local
$T \leftarrow \mathsf{Wat}'_{\mathsf{sk}}(C, Q)$ ▷ Embed via L-bit primitive
return T

6) Hierarchical Dynamic Embedding: See Algorithm 5.

Algorithm 5 Hierarchical Dynamic Scheme: Wat (Embedding)

$C_g \leftarrow X_g[g]$ ▷ Retrieve group codeword dynamically
$C_u \leftarrow \mathrm{PRF}(K_g[g], \text{“local”} \| T_u[u])$ ▷ Generate local codeword dynamically
$C \leftarrow C_g \| C_u$
$T \leftarrow \mathsf{Wat}'_{\mathsf{sk}}(C, Q)$ ▷ Embed via L-bit primitive
return T

7) Hi-DyPa Embedding: See Algorithm 6.
8) Hierarchical Static Tracing: See Algorithm 7
9) Hierarchical Dynamic Tracing: See Algorithm 8
10) Hi-DyPa Tracing: See Algorithm 9

F. Security Proofs

1) Undetectability Proof:

Proof. Undetectability, as mentioned in recent watermarking works [8], [21], [18], is preserved because Hi-DyPa's embedding mechanism does not modify the underlying watermarking primitive. The embedding algorithm $\mathsf{Wat}_{\mathsf{sk}}((g,u), Q)$ constructs the composite codeword and invokes the underlying multi-user scheme's embedding function $\mathsf{Wat}'_{\mathsf{sk}}(C, Q)$. Since the hierarchical structure only affects how the embedded message is interpreted during tracing—not how it is embedded—

Algorithm 7 Hierarchical Static Scheme: Tracing

$\hat{m} \leftarrow \mathsf{Extract}'_{\mathsf{sk}}(\hat{T})$ ▷ Extract embedded message
if $\hat{m} = \perp$ **then**
  return $\emptyset$
end if
$\hat{m}_g \leftarrow \hat{m}[1 : L_g]$ ▷ Group part
$\hat{m}_u \leftarrow \hat{m}[L_g + 1 : L]$ ▷ Local part
▷ Phase 1: Group-level tracing
$g^* \leftarrow \perp$
for $g \in [G]$ **do**
  $S \leftarrow \mathsf{FP.Trace}(\hat{m}_g, X_g[g], \mathsf{tk}_g)$
  if $S \neq \emptyset$ **then**
    $g^* \leftarrow g$
    break
  end if
end for
if $g^* = \perp$ **then**
  return $\emptyset$
end if
▷ Phase 2: User-level tracing
$u^* \leftarrow \perp$
for $u \in [s]$ **do**
  $\text{score}_u \leftarrow \mathsf{Compare}(\hat{m}_u, X_u[u], \mathsf{tk}_u)$
  if $\text{score}_u > \tau_u$ **then**
    $u^* \leftarrow u$
    break
  end if
end for
if $u^* \neq \perp$ **then**
  return $\{g^*, u^*\}$
else
  return $\{g^*\}$
end if

Algorithm 8 Hierarchical Dynamic Scheme: Tracing

$\hat{m} \leftarrow \mathsf{Extract}'_{\mathsf{sk}}(\hat{T})$ ▷ Extract embedded message
if $\hat{m} = \perp$ **then**
  return $\emptyset$
end if
$\hat{m}_g \leftarrow \hat{m}[1 : L_g]$ ▷ Group part
$\hat{m}_u \leftarrow \hat{m}[L_g + 1 : L]$ ▷ Local part
▷ Phase 1: Group-level tracing (dynamic regeneration)
$g^* \leftarrow \perp$
for $g \in [G]$ **do**
  $C_g \leftarrow \mathsf{PRF}(\mathsf{tk}_g \,\|\, K_g[g])$
  $S \leftarrow \mathsf{FP.Trace}(\hat{m}_g, C_g, \mathsf{tk}_g)$
  if $S \neq \emptyset$ **then**
    $g^* \leftarrow g$
    break
  end if
end for
if $g^* = \perp$ **then**
  return $\emptyset$
end if
▷ Phase 2: User-level tracing (dynamic regeneration)
$u^* \leftarrow \perp$
for $u \in [s]$ **do**
  $C_u \leftarrow \mathsf{PRF}(\mathsf{tk}_u \,\|\, K_g[g^*] \,\|\, T_u[u])$
  $\text{score}_u \leftarrow \mathsf{Compare}(\hat{m}_u, C_u, \mathsf{tk}_u)$
  if $\text{score}_u > \tau_u$ **then**
    $u^* \leftarrow u$
    break
  end if
end for
if $u^* \neq \perp$ **then**
  return $\{g^*, u^*\}$
else
  return $\{g^*\}$
end if

any distinguisher against Hi-DyPa directly implies a distinguisher against the base scheme $\mathcal{W}'$.

We establish undetectability through a black-box reduction to the underlying multi-user watermarking scheme of Cohen et al. [8]. Suppose $\mathcal{A}$ is an efficient adversary that distinguishes Hi-DyPa outputs from Model outputs with non-negligible advantage ε. We construct an adversary $\mathcal{B}$ against $\mathcal{W}'$ as follows: given oracle access $\mathcal{O}(\cdot,\cdot)$ which is either $\mathsf{Model}'(\cdot,\cdot)$ or $\mathsf{Wat}'_{\mathsf{sk}}(\cdot,\cdot)$, for each query $((g,u),Q)$ from $\mathcal{A}$, adversary $\mathcal{B}$ computes the group codeword, generates the local codeword, forms the composite codeword, queries $T \leftarrow \mathcal{O}(C,Q)$, and returns T to $\mathcal{A}$.

When $\mathcal{O} = \mathsf{Model}'$, adversary $\mathcal{B}$ queries $\mathsf{Model}'(C,Q)$ which returns $\mathsf{Model}(Q)$, perfectly simulating $\mathcal{A}^{\mathsf{Model}'}$. When $\mathcal{O} = \mathsf{Wat}'_{\mathsf{sk}}$, adversary $\mathcal{B}$ queries $\mathsf{Wat}'_{\mathsf{sk}}(C,Q)$ with the composite codeword, which is exactly what Hi-DyPa's embedding does, perfectly simulating $\mathcal{A}^{\mathsf{Wat}_{\text{Hi-DyPa}}}$. The dynamic codeword generation maintains computational indistinguishability because the group seeds $K_g[g]$ are independent random λ-bit strings, and by PRF security [18] the outputs are indistinguishable from random. Thus, the codeword distribution is computationally identical to random codewords. By the undetectability of the underlying scheme $\mathcal{W}'$ [18], [8], we have $\mathsf{Adv}_{\mathcal{B}} < \mathsf{negl}(\lambda)$, and therefore $\mathsf{Adv}_{\mathcal{A}} = \mathsf{Adv}_{\mathcal{B}} < \mathsf{negl}(\lambda)$. □

2) Soundness:

Proof. The soundness of Hi-DyPa follows directly from the soundness of the underlying multi-user watermarking scheme $\mathcal{W}'$ [8]. Let $T \in \mathcal{T}^*$ be any text of polynomial length $|T| \leq \mathsf{poly}(\lambda)$. We show that Hi-DyPa's detection algorithm returns 1 with negligible probability on unwatermarked text.

The Hi-DyPa detection algorithm first extracts the embedded message $\hat{m} \leftarrow \mathsf{Extract}'_{\mathsf{sk}}(\hat{T})$. If extraction returns all erasures, detection immediately outputs 0. Otherwise, the algorithm splits the message into group and user portions, applies fingerprinting code tracing on the group portion, and if group tracing succeeds, attempts user-level matching. By the soundness property of the underlying scheme $\mathcal{W}'$, which

Algorithm 9 Hi-DyPa Scheme: Tracing

$B \leftarrow \text{EXTRACTBLOCKS}(\hat{T})$ ▷ Extract high-entropy blocks
$m \leftarrow |B|$
▷ **Phase 1: Group-level tracing with parallel reconstruction**
$g^* \leftarrow \perp$
$\hat{m}_u^* \leftarrow \perp$
for $g \in [G]$ **do**
  $\mathsf{bits}_g \leftarrow [\,]$; $\mathsf{bits}_u \leftarrow [\,]$ ▷ Reconstruct layer assignments using group seed
  for $i = 1$ **to** m **do**
    $a \leftarrow \mathsf{PRF}(K_g[g], \texttt{"assign"} \| H[i])$
    $\mathsf{rnk} \leftarrow \mathsf{PRF}(K_g[g], \texttt{"rank"} \| H[i])$
    if $a < r \cdot 2^\lambda$ **then**
      $\mathsf{bits}_g.\mathsf{append}((\mathsf{raw\_bits}[i], \mathsf{rnk}))$
    else
      $\mathsf{bits}_u.\mathsf{append}((\mathsf{raw\_bits}[i], \mathsf{rnk}))$
    end if
  end for
  Sort bits_g, bits_u by rank ascending
  $\hat{m}_g \leftarrow \text{RECONSTRUCT}(\mathsf{bits}_g, L_g)$
  $\hat{m}_u \leftarrow \text{RECONSTRUCT}(\mathsf{bits}_u, L_u)$
  $S \leftarrow \mathsf{FP.Trace}(\hat{m}_g, X_g[g], \mathsf{tk}_g)$
  if $S \neq \emptyset$ **then**
    $g^* \leftarrow g$
    $\hat{m}_u^* \leftarrow \hat{m}_u$ ▷ Store user-part reconstruction for accused group
    break
  end if
end for
if $g^* = \perp$ **then**
  return $\emptyset$
end if
▷ **Phase 2: User-level tracing (dynamic regeneration)** (see Phase 2 in Hierarchical Dynamic scheme Tracing algorithm 8

inherits from the zero-bit [18] and is preserved in the multi-user extension [8], we have $\Pr[\mathsf{Extract}'_{\mathsf{sk}}(T) \neq \perp^L] < \mathsf{negl}(\lambda)$ for any unwatermarked text T.

Since detection can only succeed if extraction produces a non-trivial output, and the additional hierarchical checks (fingerprinting code tracing and user-level matching) can only reject more texts rather than falsely accept them, we obtain:

$$\Pr[\mathsf{Detect}_{\text{Hi-DyPa}}(T) = 1] \leq \Pr[\mathsf{Extract}'_{\mathsf{sk}}(T) \neq \perp^L] < \mathsf{negl}(\lambda).$$

This establishes that Hi-DyPa is sound. □

3) Completeness Analysis:

Proof. Completeness follows from the completeness of the underlying multi-user scheme [8] combined with the properties of hierarchical decoding and parallel embedding. When embedding a watermark for user (g, u), the algorithm retrieves the group codeword, generates the local codeword, forms the composite codeword, and invokes the underlying embedding $T \leftarrow \mathsf{Wat}'_{\mathsf{sk}}(C, Q)$.

The Hi-DyPa parallel embedding mechanism distributes bits across high-entropy positions, with expected group bit coverage $r \cdot m$ and user bit coverage $(1 - r) \cdot m$ for m high-entropy positions. For sufficiently large m, both layers receive proportional coverage with overwhelming probability: $\Pr[|L_G - r \cdot m| > \varepsilon \cdot r \cdot m] < 2 \cdot \exp(-\varepsilon^2 \cdot r \cdot m/3)$, and similarly for user bits.

For the generated text $T = \mathsf{Wat}_{\mathsf{sk}}((g, u), Q)$, extraction yields $\hat{m} \leftarrow \mathsf{Extract}'_{\mathsf{sk}}(T)$. By the completeness of the underlying scheme $\mathcal{W}'$ established in [8], we have $\hat{m} \in B_\delta(C)$ with probability $\geq 1 - \mathsf{negl}(\lambda)$. Splitting the extracted message gives $\hat{m}_g \in B_\delta(C_g) = B_\delta(X_g[g])$ for group-level tracing, and $\hat{m}_u \in B_\delta(C_u)$ for user-level tracing. By the completeness of the SPC code [37], group-level tracing succeeds: $\mathsf{FP.Trace}(\hat{m}_g, X_g, \mathsf{tk}) \ni g$. Once group g is identified, the user codeword is regenerated, and since $\hat{m}_u \in B_\delta(C_u)$, user-level comparison succeeds with high probability. The content-stable hashing $H[i] = \mathsf{Hash}(\mathsf{normalize}(B[i]))$ ensures deterministic layer assignments that match between embedding and tracing. Combining all steps yields $\Pr[\mathsf{Trace}_{\mathsf{sk}}(T) \ni (g, u)] \geq (1 - \mathsf{negl}(\lambda))^3 = 1 - \mathsf{negl}(\lambda)$. □

4) Adaptive Robustness:

Proof. Hi-DyPa inherits the AEB-style robustness guarantees from the underlying multi-user scheme [8], where detection succeeds whenever sufficiently high-entropy blocks survive text modifications. Following Cohen et al. [8], the robustness condition R_k evaluates to 1 when the generated text T has sufficient empirical entropy and the modified text $\hat{T}$ preserves enough of the original structure.

By the robustness of $\mathcal{W}'$ established in [8, Theorem 6.4], we have $\Pr[\mathsf{Extract}'_{\mathsf{sk}}(\hat{T}) \in B_\delta(C) \mid R_k = 1] \geq 1 - \mathsf{negl}(\lambda)$. The parallel embedding mechanism provides additional robustness advantages over sequential embedding. Under sequential embedding with deletion ratio d at position p, group or user bits can be destroyed depending on the deletion location. Under parallel embedding, for any deletion position, we have $\mathbb{E}[f_g] \approx r \cdot (1 - d)$ and $\mathbb{E}[f_u] \approx (1 - r) \cdot (1 - d)$, where f_g and f_u denote the fraction of surviving group and user bits, respectively.

The content-stable hashing ensures that minor text modifications within a block do not change the hash value, layer assignments remain stable under paraphrasing, and PRF-based layer determination is deterministic given the group seed. When $R_k = 1$, sufficient blocks survive, the extracted message $\hat{m} = \mathsf{Extract}'_{\mathsf{sk}}(\hat{T}) \in B_\delta(C)$, and detection succeeds. Therefore $\Pr[\mathsf{Detect}_{\mathsf{sk}}(\hat{T}) = 1 \mid R_k = 1] \geq 1 - \mathsf{negl}(\lambda)$, prove that Hi-DyPa is adaptively robust. □

5) Collusion Resistance:

Proof. Hi-DyPa achieves collusion resistance through two complementary mechanisms: group-level fingerprinting codes that provide mathematical guarantees for identifying at least

one guilty group [37], and hierarchical structure that reduces the collusion problem to smaller subproblems. At any bit position i where all colluders share the same value, the adversary cannot produce a different value. This defines the feasible set $\mathcal{F}(\mathcal{T})$ of achievable colluded codewords.

By the robustness of the multi-user scheme [8], we have: $\Pr[\mathsf{Extract}'_{\mathsf{sk}}(\hat{T}) \in B_\delta(\mathcal{F}(\mathcal{T})) \mid R_k = 1] \geq 1 - \mathsf{negl}(\lambda)$. We analyze two cases. For same-group collusion where all colluders share group g^*, all colluders have identical group codeword $C_g^{(i)} = X_g[g^*]$, so the feasible set for the group layer is a singleton: $\mathcal{F}_g(\mathcal{T}) = \{X_g[g^*]\}$. The extracted group portion $\hat{m}_g \in B_\delta(X_g[g^*])$, and group g^* is identified with overwhelming probability. For user-level tracing, the dynamic codewords differ across users, and standard fingerprinting analysis applies within the group.

For cross-group collusion where colluders span multiple groups, the group codewords differ: $X_g[g_i] \neq X_g[g_j]$ for $g_i \neq g_j$. By the proven collusion resistance of the SPC code, at least one guilty group is identified from any codeword in the feasible set. Once a group g^* is identified, user-level tracing proceeds using dynamically regenerated codewords from the accused group's seed $K_g[g^*]$, preventing cross-group framing. Combining both cases with the union bound yields $\Pr[\mathsf{Trace}(\hat{T}) \cap \mathcal{T} \neq \emptyset \mid R_k = 1] \geq 1 - \mathsf{negl}(\lambda)$. □

6) Framing Attack Resistance:

Proof. Let $\mathcal{T} = \{(g_1, u_1), \ldots, (g_c, u_c)\}$ be a coalition and $(g^*, u^*) \notin \mathcal{T}$ be the target innocent user.

When $g^* \notin \{g_1, \ldots, g_c\}$ (target in a different group), the colluders have no access to $K_g[g^*]$. By PRF security [18], the target's local codeword is computationally indistinguishable from a uniformly random L_u-bit string. The probability of framing is therefore $\Pr[\text{colluders guess } C_u^{(g^*,u^*)}] \leq 2^{-L_u} + \mathsf{negl}(\lambda)$, which is negligible for any L_u.

When $g^* \in \{g_1, \ldots, g_c\}$ but $u^* \notin \{u_i : g_i = g^*\}$ (target in same group, different user position), colluders know some codewords from group g^* but the target's codeword uses a different template $T_u[u^*] \neq T_u[u_i]$ for all colluders u_i. Since PRF inputs differ, the outputs are computationally independent, and this reduces to standard fingerprinting code security within the group.

We establish the result through reduction to PRF security. Suppose adversary $\mathcal{A}$ frames innocent (g^*, u^*) with probability ε. We construct a PRF distinguisher $\mathcal{B}$ that generates $K_g[g]$ normally for colluder groups and uses a PRF oracle $\mathcal{O}$ for group g^*. If $\mathcal{A}$ successfully frames (g^*, u^*), $\mathcal{B}$ outputs "real PRF"; otherwise $\mathcal{B}$ outputs "random." When $\mathcal{O} = \mathsf{PRF}(K^*, \cdot)$, $\mathcal{A}$'s view is identical to real Hi-DyPa. When $\mathcal{O}$ is random, the target codeword is truly random and $\Pr[\mathcal{A} \text{ frames } (g^*, u^*)] \leq 2^{-L_u}$. The distinguishing advantage is $\mathsf{Adv}_{\mathcal{B}} = |\varepsilon - 2^{-L_u}|$. By PRF security, $\mathsf{Adv}_{\mathcal{B}} < \mathsf{negl}(\lambda)$, so $\varepsilon < 2^{-L_u} + \mathsf{negl}(\lambda) = \mathsf{negl}(\lambda)$. □

G. Tracing Time Complexity Analysis Details

We prove that the tracing-time complexity of the Hierarchical Static scheme is no greater than that of the flat multi-user baseline. Let

$$\mathcal{T}_{\text{flat}}(n) \;=\; n \cdot L$$

denote the tracing-time complexity of the flat multi-user baseline, and let

$$\mathcal{T}_{\text{hier}}(G, s) \;=\; G \cdot L_g + s \cdot L_u$$

denote the tracing-time complexity of the Hierarchical Static scheme, where $G, s \in \mathbb{Z}_{\geq 1}$, $n = G \cdot s$, $L = L_g + L_u$, and $L_g, L_u > 0$. Then:

$$\mathcal{T}_{\text{hier}}(G, s) \;\leq\; \mathcal{T}_{\text{flat}}(n),$$

with equality if and only if $n = 1$.

The comparison terms satisfy

$$G \cdot L_g + s \cdot L_u \;\leq\; n \cdot L. \tag{2}$$

Substituting $n = G \cdot s$ and $L = L_g + L_u$ into the right-hand side of (2) yields

$$n \cdot L \;=\; G \cdot s \cdot (L_g + L_u) \;=\; G \cdot s \cdot L_g + G \cdot s \cdot L_u.$$

Define $\Delta := n \cdot L - (G \cdot L_g + s \cdot L_u)$. Subtracting and regrouping the L_g- and L_u-terms gives the identity

$$\Delta \;=\; G \cdot L_g \cdot (s - 1) \;+\; s \cdot L_u \cdot (G - 1). \tag{3}$$

Each summand of (3) is a product of non-negative quantities: $G, s \geq 1$ implies $(G-1), (s-1) \geq 0$, and $G, s, L_g, L_u > 0$ by assumption. Hence $\Delta \geq 0$, which establishes (2) and therefore (G).

The equality condition $\Delta = 0$ requires both summands of (3) to vanish simultaneously. Since $G \cdot L_g > 0$ and $s \cdot L_u > 0$, this forces $s = 1$ and $G = 1$, hence $n = G\,s = 1$. For every $n \geq 2$, at least one of $G > 1$ or $s > 1$ holds, so at least one summand of (3) is positive and $\Delta > 0$.

H. Bit Budget and Attribution Capacity

1) Bit-Budget Constraints in Language-Model Watermarking: Multi-bit watermarking in language models operates under a limited bit budget. Unlike classical fingerprinting settings where long codes can be spread across many independent samples, watermark recovery in language model outputs relies on finite-length generations, where each bit is supported by only a small number of watermark-bearing tokens. As the number of embedded bits increases, the statistical evidence available for each bit decreases. Prior empirical studies [17], [46] consistently show that reliable recovery is achievable only for relatively small message lengths, with performance degrading rapidly as the bit budget grows. This behavior reflects finite-sample effects rather than limitations of a specific embedding or detection method. As a result, attribution schemes for language models must operate within a small and fixed number of reliably decodable bits; in this work, we treat the watermark length L as a constrained design parameter and study how attribution capacity can be structured under this constraint.

2) Hierarchical Attribution Capacity: Under Hi-DyPa, a recovered L-bit watermark is interpreted hierarchically at decoding time. The message is conceptually partitioned into L_g group bits and $L_u = L - L_g$ user bits, identifying a coarse attribution group and an individual user within that group, respectively. This partitioning is applied only at decoding time; embedding and detection operate on the full L-bit message.

To ensure robustness to cross-group ambiguity, group identifiers are drawn from a codeword satisfying a minimum Hamming distance of two. Under this constraint, L_g bits support up to 2^{L_g-1} distinct groups, while the remaining L_u bits encode up to 2^{L_u} users within each group. The total number of supported users under hierarchical decoding is therefore

$$2^{L_g-1} \cdot 2^{L_u} = 2^{L-1},$$

which depends only on the total bit budget L and not on the specific (L_g, L_u) allocation. The motivation for imposing this minimum-distance group constraint, and its implications for robustness and attribution behavior under noise, are discussed in Appendix H3.

TABLE XVIII: Hierarchical attribution capacity as a function of watermark length L. Total supported users follow 2^{L-1} under the minimum-distance group constraint.

L	**Total Supported Users** (2^{L-1})
4	8
5	16
6	32
7	64
8	128
9	256
10	512
11	1024
12	2048

Table XVIII summarizes the resulting attribution capacity as a function of L in our setting. While increasing L exponentially increases the number of representable users, operating at larger L requires reliable multi-bit recovery from limited-length model outputs. The empirical feasibility of different L values is evaluated separately in Appendix I3.

3) Graceful Degradation and Partial Attribution: To implement the minimum-distance constraint on group identifiers, we use a *single parity-check* (SPC) code over the L_g group bits, i.e., the set of all even-parity binary strings, widely used as the simplest high-rate component in modern product and concatenated codes [53], [37]. This construction forms a linear $[L_g, L_g - 1, 2]$ code with 2^{L_g-1} valid codewords and minimum Hamming distance two. The SPC code is the simplest mechanism that enforces inter-group separation while preserving the maximum possible number of groups under a fixed bit budget.

This design explicitly prioritizes reliability at the group level. Because distinct groups differ by at least two bits, a single-bit error in multi-bit recovery produces an invalid group label rather than a competing valid group, allowing the decoder to abstain from cross-group attribution. In contrast, flat decoding over all 2^{L-1} users concentrates uncertainty into a single decision, increasing the risk of confident misattribution under noise.

As a result, hierarchical decoding enables graceful degradation. When watermark evidence is insufficient to support confident user identification, group-level attribution may remain reliable, effectively narrowing the attribution space without committing to an incorrect individual identity. While our evaluation fixes a strict single-identity output for conservatism and comparability, this structure naturally supports group-only or small candidate-set attribution strategies, which we leave to future work. An empirical breakdown of attribution outcomes across hierarchical stages, illustrating how group- and user-level accuracies interact under clean detection, is provided in Appendix I5.

4) Trade-offs in Group and User Bit Allocation: For a fixed watermark length L, all hierarchical partitions (L_g, L_u) with $L_g + L_u = L$ support the same total attribution capacity under the minimum-distance group constraint (Appendix H2). As a result, varying the allocation between group and user bits does not change capacity, but instead redistributes where statistical uncertainty, decoding effort, and attribution resolution are concentrated within the hierarchy. The (L_g, L_u) split therefore governs attribution behavior under noise and decoding structure, rather than determining how many users can be supported.

5) Decoding Trade-offs and Attribution Semantics.: Although total attribution capacity is conserved, the (L_g, L_u) split determines how uncertainty is distributed during decoding and how attribution semantics are expressed. Allocating more bits to group identification increases robustness at a coarse attribution level but reduces resolution among users within a group, while allocating more bits to user identification sharpens within-group resolution at the cost of reduced group-level stability. This trade-off naturally aligns with common organizational settings in which users belong to shared units such as teams or roles. In such cases, prioritizing group bits enables reliable localization of responsibility at the organizational level even when individual-level evidence is weak, whereas prioritizing user bits emphasizes fine-grained attribution within a known group. Hi-DyPa exposes this design choice explicitly, allowing attribution structure to reflect operational goals rather than treating all users as unrelated flat identities.

6) Decoding Complexity and Error Allocation.: Under a fixed watermark length $L = L_g + L_u$, hierarchical decoding does not change the overall exponential dependence on L, but it redistributes decoding cost across stages. When $L_u \gg L_g$, user-level resolution dominates the decoding cost, yielding complexity on the order of $O(2^{L_u})$, whereas when $L_g \gg L_u$, group-level resolution dominates with complexity $O(2^{L_g-1})$. For balanced splits where $L_g \approx L_u$, decoding costs are distributed more evenly across stages. Importantly, these comparisons are meaningful only under fixed L and reflect where uncertainty and computational effort are absorbed, rather than changes in total attribution capacity. By resolving group identity first, hierarchical decoding allows expensive user-level decisions to be skipped when group evidence is

weak, improving practical attribution behavior under noisy multi-bit recovery.

I. Extended Experimental Results

TABLE XIX: Computational overhead (per run) (GPT-2): initialization time, embedding time, detection time (seconds), and additional metadata storage (KB).

Scheme	(L_g, L_u)	Init (s)	Embed (s)	Detect (s)	Storage (KB)
MAU [8]	(0,8)	0.090	5.640	1.863	0.00
Hi-DyPa	(1,7)	0.194	4.974	1.826	0.07
Hi-DyPa	(2,6)	0.101	4.728	1.768	0.15
Hi-DyPa	(3,5)	0.069	4.704	**1.746**	0.31
Hi-DyPa	(4,4)	0.082	4.707	1.766	0.63
Hi-DyPa	(5,3)	0.074	4.740	1.784	1.28
Hi-DyPa	(6,2)	0.058	4.806	1.773	2.59
Hi-DyPa	(7,1)	**0.054**	**4.694**	1.763	5.25

TABLE XX: Computational overhead (per run) (DeepSeek 7B): initialization time, embedding time, detection time (seconds), and additional metadata storage (KB).

Scheme	(L_g, L_u)	Init (s)	Embed (s)	Detect (s)	Storage (KB)
MAU [8]	(0,8)	0.067	8.967	1.391	0.00
Hi-DyPa	(1,7)	0.061	8.585	1.338	0.05
Hi-DyPa	(2,6)	0.061	8.905	1.362	0.11
Hi-DyPa	(3,5)	0.062	8.439	1.314	0.22
Hi-DyPa	(4,4)	0.062	8.568	1.339	0.44
Hi-DyPa	(5,3)	0.062	8.797	1.368	0.88
Hi-DyPa	(6,2)	0.119	**8.393**	1.303	1.75
Hi-DyPa	(7,1)	**0.061**	8.413	**1.299**	3.50

1) Additional System Computational Overhead Evaluation: The efficient storage improvement comes at the cost of a small amount of additional auxiliary storage for precomputed group codewords, which grows with the number of groups but remains modest (at most ≈5.25 KB in our implementation). Overall, Hi-DyPa significantly accelerates tracing while maintaining comparable embedding and detection costs, with only a minimal and controlled increase in memory usage.

2) Additional Framing Attacks Results: For the CRR results of different colluding trials k, see figure 3.

3) Bit Budget Selection: We study watermark messages of fixed length L, embedded as a single concatenated bit string using a shared embedding procedure. Across all schemes, the detector recovers the full L-bit message using the standard AEB multi-bit recovery procedure. For Hi-DyPa, the recovered bits are subsequently interpreted hierarchically at decoding time; the embedding and detection primitives themselves are identical across configurations.

To empirically assess the reliability of multi-bit recovery, we perform an *L-bit sweep* varying L from 4 to 30 while holding all other parameters fixed. For each prompt, a random L-bit message is embedded and subsequently recovered by the detector. For a given prompt, bit recovery accuracy is defined as the fraction of bits that are correctly recovered, counting undecided bits as incorrect. We report the *average accuracy* for each L as the mean of this quantity across prompts.

We fix the watermark bit budget to $L = 8$ for all subsequent experiments. As shown in Table XXI, bit recovery accuracy remains high for small values of L but degrades rapidly as L increases, with a pronounced drop beyond $L \approx 10$. At $L = 8$, average bit recovery accuracy remains above 90%, providing a stable operating point for all attribution experiments reported in the main paper.

TABLE XXI: L-bit sweep on GPT-2 using 300 prompts. Average accuracy is the mean per-prompt bit recovery accuracy.

L	Avg. Accuracy (%)	Exact Match Rate (%)	Avg. Undecided Bits
4	98.92	97.67	0.04
5	97.60	95.33	0.12
6	95.56	92.67	0.27
7	95.29	92.00	0.33
8	91.71	83.67	0.66
9	91.00	80.00	0.81
10	82.80	55.67	1.72
11	80.85	35.33	2.11
12	72.28	10.33	3.33

4) Watermark Parameter Selection and Qualitative Analysis: **Role of Embedding Strength δ and Entropy Threshold τ.** Watermark embedding in our AEB-style framework is controlled by two parameters: the embedding strength δ and the entropy threshold τ. The parameter δ determines the magnitude of the bias applied to watermark-favored tokens whenever embedding is active, with larger values strengthening the watermark signal but risking distortion of the sampling distribution. The entropy threshold τ governs where embedding is applied by restricting watermark insertion to token positions whose next-token distribution entropy exceeds τ, thereby trading off embedding frequency against generation quality. Together, δ and τ control the balance between watermark detectability and output quality, following standard entropy-gated watermarking practice [9], [18], [11], [54], [55].

Parameter Sweep and Candidate Selection. To select a fixed and robust watermark configuration for all subsequent experiments, we perform a controlled sweep over the embedding strength δ and entropy threshold τ while holding all other parameters constant. All runs use GPT-2 and Deepseek-7B with a fixed watermark length $L = 8$, identical detector settings, and a fixed generation setup consistent with Section VI-A. The sweep considers $\delta \in \{2.0, 2.5, 3.0, 3.5, 4.0\}$ and $\tau \in \{1.5, 2.0, 2.5, 3.0, 3.5\}$, yielding 25 parameter combinations in total. The evaluation is conducted on the same set of 300 prompts used for all other experiments. For each prompt and each (δ, τ) pair, a random L-bit watermark message and key are sampled, embedded, and recovered by the detector. We compute the per-prompt bit recovery accuracy as the fraction of correctly recovered bits, counting undecided bits as incorrect, and report the average accuracy across prompts for each parameter configuration.

Table XXII reveals a clear separation between weak and strong watermarking regimes. Configurations with low embedding strength ($\delta = 2.0$) perform poorly across all entropy thresholds, indicating that insufficient bias fails to produce a reliably decodable signal even when embedding is applied frequently. In contrast, moderate to high values of δ yield consistently high recovery accuracy across a range of entropy thresholds, forming a broad performance plateau rather than

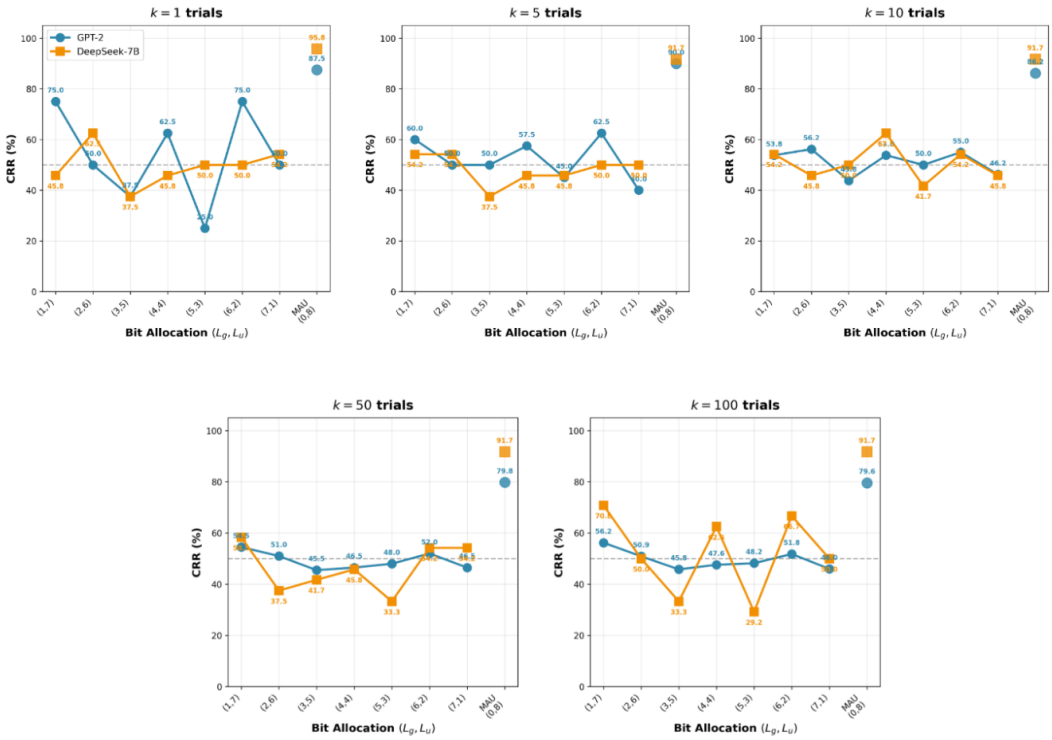

Fig. 3: Codeword Recovery Rate (CRR) across different adversarial colluding trials

TABLE XXII: Representative results from the (δ, τ) parameter sweep at $L = 8$. We report the top three best-performing and bottom three worst-performing configurations by average bit recovery accuracy from the full 25-point grid.

δ	Entropy Threshold τ	Avg. Accuracy (%)
Best-performing configurations		
4.0	**2.5**	**97.63**
4.0	3.5	96.88
3.5	2.0	94.88
Worst-performing configurations		
2.0	2.0	62.75
2.0	2.5	62.50
2.0	3.5	58.00

a sharp optimum. This behavior reflects the complementary roles of δ and τ: once the watermark signal is sufficiently strong, detection accuracy becomes less sensitive to the precise frequency of embedding. As a result, several (δ, τ) pairs achieve similarly high quantitative performance. However, bit recovery accuracy alone does not capture the impact of watermarking on generation quality, and configurations with comparable detection performance can differ substantially in fluency, repetition, and semantic coherence. We therefore perform qualitative inspection of candidate configurations to select a final operating point for all subsequent experiments.

Qualitative Validation.

Based on the quantitative sweep in the previous section, we select the top three parameter configurations by average bit recovery accuracy for qualitative inspection: $(\delta, \tau) \in \{(4.0, 2.5), (4.0, 3.5), (3.5, 2.0)\}$. These configurations achieve comparable detection performance and differ primarily in the strength and frequency of watermark embedding. To assess their impact on generation quality, we perform controlled qualitative inspection using a fixed prompt drawn from the FLAN [42] instruction-following dataset ("*The future of AI is*"), holding the base model (GPT-2), decoding procedure, generation length, and all non-watermark parameters constant across runs. For each configuration, we examine the initial portion of the generated response under identical conditions. This inspection is not intended as a formal linguistic evaluation, but as a qualitative sanity check to identify configurations that visibly disrupt semantic coherence despite strong quantitative detection performance.

$(\delta, \tau) = (4.0, 2.5)$

```
The future of AI is hard's full release
since there are billions of lives most
could work with any OC developers could
enter that room managing this....
```

$(\delta, \tau) = (4.0, 3.5)$

```
The future of AI is unclear with AI
being able to get foreign links on push
accounts and expiration check to
enticks detection...
```

$(\delta, \tau) = (3.5, 2.0)$

```
The future of AI is full of miracles;
they make, visualize, embody, connect,
transhumanize future members. Will Data
supply human virtually?...
```

For completeness, we also include an illustrative example using the same prompt under a conservative watermarking configuration $(\delta, \tau) = (1.5, 1.5)$.

$(\delta, \tau) = (1.5, 1.5)$

```
The future of AI is constantly
evolving, with lots of new ways of
building artificial intelligence. And
if AI succeeds tomorrow, we will have
the same goal: better food and
health...
```

While GPT-2 outputs are not expected to be highly polished, relative differences across configurations remain visually apparent under identical conditions. The outputs reveal clear qualitative differences among configurations despite their comparable quantitative detection performance. For $(\delta, \tau) = (4.0, 2.5)$ and $(4.0, 3.5)$, the generated text exhibits visible semantic disruption, including malformed constructions, abrupt topic shifts, and incomplete or fragmented clauses, indicating that overly aggressive watermark bias can interfere with confident next-token predictions. Although these configurations achieve strong bit recovery accuracy, the resulting outputs fail a basic semantic coherence sanity check for practical use, making them unsuitable for many downstream applications. In contrast, the configuration $(\delta, \tau) = (3.5, 2.0)$ preserves grammatical structure and produces a coherent continuation of the prompt without obvious fragmentation or semantic drift. Although $(3.5, 2.0)$ ranks slightly below the top two configurations in average bit recovery accuracy, its detection performance remains high and comparable within the observed performance plateau. This setting therefore provides a more stable balance between reliable watermark detection and acceptable generation behavior, and we fix $(\delta, \tau) = (3.5, 2.0)$ for all subsequent experiments in Section VI.

5) Extended Attribution Analysis: While the main paper reports full identity accuracy as the primary end-to-end attribution metric, this scalar alone does not reveal how attribution errors arise in hierarchical decoding, nor whether failures occur at the group level or within-group user resolution. To better understand the behavior of Hi-DyPa under clean detection evaluation in Section VI-C, we decompose attribution performance into group-level accuracy and user-level accuracy conditioned on correct group identification. Reporting these metrics jointly allows us to characterize how attribution degrades across hierarchical stages and to assess whether partial attribution remains informative when full identity recovery fails. We include full identity accuracy and false positive rates here only as a reference point to contextualize the relationship between hierarchical decisions and end-to-end outcomes, rather than to re-evaluate overall detection performance. This decomposition directly reflects the hierarchical design choices discussed in Appendix H3, and motivates group-level attribution as a meaningful fallback when full identity recovery fails.

Table XXIII illustrates how attribution performance decomposes across the two hierarchical decoding stages. As the number of group bits L_g decreases, group accuracy improves due to the reduced number of candidate groups and increased inter-group separation, while user accuracy correspondingly decreases because a larger user space must be resolved within each group. Conversely, allocating more bits to group identification (larger L_g) reduces group accuracy but substantially improves conditional user accuracy by shrinking the within-group user space. Across all configurations, full identity accuracy closely follows the product of group and user accuracy, indicating that overall attribution performance is governed by the interaction between coarse group resolution and conditional user identification within that group. We note that false positives are computed under a single-identity attribution protocol; alternative designs that return a small candidate set (e.g., top-k users or group-level attribution only) could trade strict false positives for partial but informative attribution.

TABLE XXIII: Hierarchical attribution performance on clean text at $L = 8$

(L_g, L_u)	Group Acc.	User Acc.	Full ID Acc.	FP
(1,7)	0.940	0.950	0.893	0.047
(2,6)	**0.950**	0.930	0.883	0.060
(3,5)	0.943	0.951	0.897	0.047
(4,4)	0.947	0.961	0.910	0.057
(5,3)	0.913	0.982	0.897	0.060
(6,2)	0.927	**0.986**	**0.913**	**0.030**
(7,1)	0.897	0.981	0.880	0.037

J. Future Work

1) System-Level Overhead and Scalability: The memory overhead introduced by hierarchical decoding is minimal in our current setting, requiring only a few kilobytes to store group and user codewords. While this overhead is negligible relative to model and generation costs at the evaluated scale, a more detailed system-level analysis is needed to assess scalability under larger user populations, longer-running deployments, and different implementation choices.

2) Partial Attribution and Graceful Degradation: Our evaluation enforces a strict single-identity output for attribution, even when the recovered signal is weak. However, hierarchical decoding naturally supports more conservative behavior: group identification may remain reliable even when user-level attribution fails. This enables a form of graceful degradation, where the system could return group-only attribution or abstain from user-level decisions rather than forcing a potentially incorrect identity. Designing and evaluating such partial attribution interfaces is an important direction for future work (see Appendix H3 and Appendix I5).

3) Scaling the Bit Budget: The L-bit sweep shows that reliable recovery degrades beyond low-teen bit lengths in our GPT-2 setup, motivating the use of grouping under a fixed $L = 8$ budget. An open question is whether larger models and/or longer generations support reliable decoding at higher L, enabling increased attribution capacity within the same framework. Exploring this scaling behavior and its interaction with hierarchical bit allocation is left to future work.

4) Domain-Specific Deployment and Factuality Constraints: Recent work has evaluated watermarking methods in

medical text generation, where attribution mechanisms must preserve factual correctness in addition to fluency. Hastuti et al. [56] show that several existing watermarking approaches can negatively impact medical factuality, particularly when watermarking perturbs low-entropy tokens that carry domain-critical information. These findings motivate future evaluation of hierarchical attribution schemes under domain-specific constraints, including stricter tolerance for factual errors and the potential role of conservative behaviors such as abstention or group-level attribution in high-stakes deployment settings.

5) Layer-wise Design for Hierarchical Watermarking Scheme: While Hi-DyPa currently employs a two-level hierarchy, the framework can achieve scalability by extending to multi-layer nested structures that mirror real-world organizational depth. This extensions would distribute the L-bit budget across multiple layers, enabling logarithmic-depth tracing for larger user populations while maintaining storage efficiency. Multi-layer configurations would also support flexible partial attribution, returning progressively specific organizational units as watermark evidence permits. This represents a promising direction for large-scale enterprise deployments with deeper hierarchical structures.